



Research paper

Intelligent congestion control in 5G URLLC Software-Defined Networks using adaptive resource management via Reinforced Dueling Deep Q-Networks

Vitawat Sittakul^a, Iacovos Ioannou^{b c}  , Prabagarane Nagaradjane^d, Vasos Vassiliou^{b c}[Show more](#)  Outline |  Share  Cite<https://doi.org/10.1016/j.jnca.2025.104276> [Get rights and content](#) [Full text access](#)

Highlights

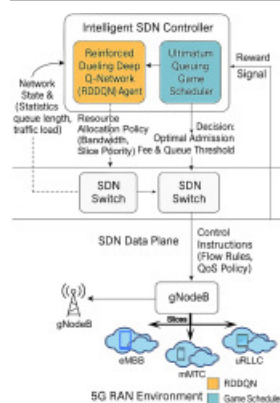
- RDDQN with ultimatum-game scheduler for real-time 5G slice congestion control.
- Learns bandwidth, admission fees and queue thresholds from live traffic statistics.
- Achieves high throughput and **> 94%** fairness, outperforming DRL/GNN/GA/ADMM baselines.
- FL enables scalable RDDQN co-training of SDN controllers without sharing raw data.
- RDDQN meets URLLC sub-5 ms control budgets; runs on commodity ONOS/OpenFlow testbeds.

Abstract

Centralized control of Software Defined Networking (SDN) yields efficient management of network resources and offers a global perspective. However, centralized controllers have many performance and scalability issues, particularly given the rapid expansion of 5G connectivity. The latest demands on the transport network come from areas such as increasing RAN and mobile broadband service capacity, new 5G-enabled services and the dynamic deployment flexibility of the 5G Radio Access Network (RAN) split architecture, with its tight transport characteristics. These characteristics are particularly evident in the fronthaul segment of RAN, where latency and synchronization requirements pose significant challenges. Enhanced automation capabilities in the operations

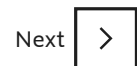
and management domain represent a key requirement to meet these challenges. Traditional machine learning (ML) techniques, which concentrate the training data and carry out sequential model learning over a sizable data set, are the main emphasis of current wireless network learning approaches. However, using a huge dataset for training is inefficient since it takes a lot of time and does not use resources or energy efficiently. Hence, this work focuses on Reinforced Dueling Deep Q-Network (RDDQN), a revolutionary approach to network slicing design for load prediction and resource management in data-driven workflows. Moreover, it can reduce congestion by adopting an Ultimatum queuing game theory-based scheduling mechanism in the controller. The proposed RDDQN achieves an average throughput of 579.34 kbps, an execution time of 12.57 s, goodput fairness of 94.56%, and delay fairness of 10.37 s across various parameters.

Graphical abstract



[Download: Download high-res image \(189KB\)](#)

[Download: Download full-size image](#)



Keywords

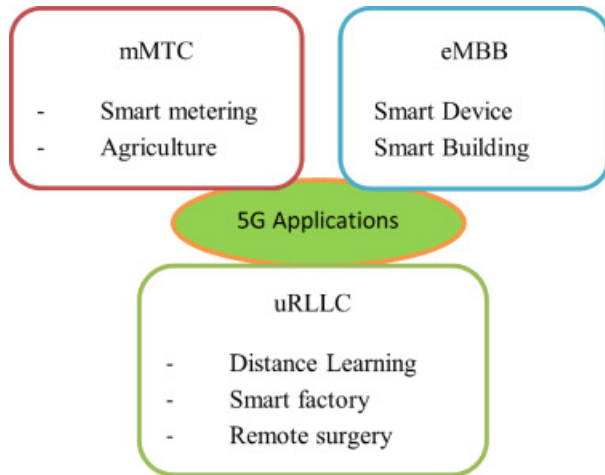
Software Defined Network; 5G communication; Game theory; Scheduling; Resource allocation; Control plane; Neural network

1. Introduction

Mobile networks are expected to support three key service categories: enhanced Mobile Broadband (eMBB), massive Machine-Type Communication (mMTC), and ultra-reliable low-latency communication (uRLLC) (Salhab et al., 2019). According to a feasibility study from the Third Generation Partnership Project (3GPP) (3GPP TR 22.864, 2016), each class imposes different and demanding throughput, mobility, dependability, latency, and energy efficiency requirements. Cost-effectively meeting these diverse needs is a significant challenge in modern 5G architectures. Fig. 1 illustrates various 5G application scenarios, which reflect the growing range of use cases and performance expectations.

Despite the promises of 5G, there is an inherent *problem* arising from resource scarcity—both infrastructural (e.g., computing, networking, storage) and exogenous (e.g., spectrum, power). Even with technological enablers like Network Function Virtualization (NFV) and Software-Defined Networking (SDN), which allow for creating separate logical networks or Network Slices (NSs) (Samdanis et al., 2017), the exponential growth in mobile traffic remains a significant challenge. An escalating volume of user and IoT traffic (Chen et al., 2018, Wang et al., 2019) can overwhelm both edge (base-station) and remote data-center resources, while distributing bandwidth across a

wide variety of applications (Abadi et al., 2015) becomes increasingly difficult. Existing data-driven approaches often offload computational tasks without analysis or coordination, which can diminish reliability and inflate latency, especially for time-sensitive traffic (Zhang et al., 2018). When heterogeneous eMBB and mMTC bursts flood the fronthaul, static threshold rules embedded in SDN controllers react too slowly, standard deep-RL allocators (e.g., vanilla DQN, PPO) converge sluggishly and incur high inference cost on the high-dimensional 5G state space, and heuristic fairness-tuning schemes such as Genetic Algorithms achieve equity only by throttling throughput—leading to transient congestion, unpredictable queue lengths, and violated uRLLC service-level agreements.



Download: [Download high-res image \(193KB\)](#)

Download: [Download full-size image](#)

Fig. 1. Application scenarios of 5G.

The *motivation* for this work stems from the realization that ultra-reliable low-latency communication demands advanced mechanisms to manage dynamic traffic flows while preserving Quality of Service (QoS). In particular, 5G networks must simultaneously accommodate latency-critical traffic (e.g., industrial automation, remote healthcare) and more conventional broadband services, all under rapidly changing conditions. Classical machine learning (ML) methods, though effective for pattern recognition, typically rely on large centralized datasets and extensive offline training, which can be impractical for real-time 5G scenarios. In contrast, modern artificial intelligence (AI) paradigms (Liang, 2019, Zhou et al., 2018) can proactively monitor and orchestrate resources, anticipating load variations through algorithms like Long Short-Term Memory (LSTM) (Feng et al., 2018) for time-series forecasting.

From the standpoint of *innovation and novelty*, our approach goes beyond simple deep reinforcement learning by embedding a *Reinforced Dueling Deep Q-Network* (RDDQN) within an *ultimatum game-theoretic* framework. This combination addresses both the performance and fairness aspects of resource allocation: RDDQN adapts rapidly to changing network states. At the same time, ultimatum game theory ensures that resource distribution among various slices is efficient and equitable. By coupling these methodologies, we can better handle mission-critical traffic slices (like URLLC) and still accommodate less stringent but high-volume flows.

Contributions of this work are summarized as follows:

- (1) **RDDQN-Based Resource Allocation:** We design a slicing mechanism leveraging a Reinforced Dueling Deep Q-Network, replacing the conventional fully connected input layer with a reinforced layer. This enhances adaptability in dynamic environments, ensuring dedicated and predictable performance for uRLLC slices.
- (2) **Ultimatum Game-Theoretic Optimization:** We solve the multi-tenancy resource allocation problem under shared constraints using an ultimatum game approach. This improves slice-level

efficiency and ensures fairness by dynamically adjusting allocations based on real-time network conditions.

- (3) **Adaptive Congestion Control at the TCP Level:** To reduce congestion and latency further, we analyze the Transmission Control Platform (TCP) for social welfare, controller profit, and switch benefit. We also determine optimal queue lengths under first-come, first-served (FCFS) and last-come, first-served (LCFS) disciplines to achieve minimal delay in critical scenarios.

Unlike standard Dueling Deep Q-Networks (Dueling DQNs), our proposed RDDQN architecture integrates a **specially designed reinforced input layer** positioned before the value and advantage streams, enabling more efficient processing of high-dimensional and volatile 5G network states. This architectural enhancement significantly boosts the adaptability and learning capacity of the model in ultra-reliable low-latency communication (URLLC) environments, where conventional reinforcement learning agents often struggle to converge rapidly. In addition to this structural innovation, we embed an **ultimatum game-theoretic scheduling mechanism directly within the RDDQN framework**, addressing the complexities of multi-tenancy and fairness in dynamic resource allocation. This game-theoretic integration, grounded in established economic principles (Güth et al., 1982), allows the model to not only learn optimal scheduling policies but also to enforce fairness constraints in a principled manner, ensuring equitable access to limited resources across multiple network slices. While the Dueling DQN (Wang et al., 2016) provides the foundational structure, our RDDQN introduces critical modifications tailored to the demands of intelligent congestion control and adaptive resource management in 5G URLLC Software-Defined Networks. The reinforced input layer enhances feature extraction and situational awareness, while the embedded ultimatum game framework governs strategic negotiation and resource partitioning among heterogeneous tenants. Together, these enhancements enable our RDDQN to address key limitations of prior methods — namely, transient congestion, unpredictable queuing dynamics, and inconsistent service-level adherence — offering a holistic and scalable solution for next-generation wireless systems.

The remainder of this article is organized as follows. Section 2 provides a literature survey on resource management in network slicing. Section 3 presents a detailed problem and system description, and Section 4 introduces our proposed method. Section 5 includes a thorough performance evaluation, followed by conclusions and future directions in Section 6.

2. Related works

This section focuses on the issues related to resource management in network slicing, seeking to tackle the most important difficulties in this field and offering a timely and thorough overview of the state of the art. In particular, we will demonstrate that cutting-edge methods in artificial intelligence and machine learning are used to support resource management in sliced wireless networks.

An autonomous Deep Reinforcement Learning agent is introduced in Boni et al. (2024) that can autonomously receive requests for slices and suggest a placement on the physical infrastructure that maximizes accepted requests while ensuring load balancing across infrastructure resources. A unique approach is proposed in Saibharath et al. (2023) for NS Resource Partitioning and User Allocation. It proposes a priority class-based packet scheduling and resource allocation system based on an online virtual backbone.

A multi-agent resource allocation approach is developed in Yuan et al. (2023) that integrates graph attention network (GAN) and deep Q network (DQL) using graph convolution reinforcement learning. Adaptively choosing the cognitive users' channel and transmit power allows it to maximize cognitive network throughput, spectrum efficiency, or power efficiency. The Twin-GAN-based DRL approach, suggested in Boni et al. (2024) by expanding the current generative adversarial network-based DRL result, uses two GAN-based DRLs to manage wireless bandwidth resources and CPU resources jointly. It seeks to decrease the usage of computing resources and increase spectrum efficiency. Twin-actor deep deterministic policy gradient, or twin-actor DDPG, is a novel DRL technique proposed in Boni et al. (2024) and is used in a way that the actor creates the deterministic policy and

the critic assesses and directs the actor to reach the best policy. An energy-efficient deep reinforcement learning-assisted resource allocation technique for Radio Access Network (RAN) slicing in 5G networks is put out in [Azimi et al. \(2021\)](#).

The stacked and bidirectional long-short-term-memory (SBiLSTM) network and the asynchronous advantage actor-critic (A3C) are employed as supervised DL and DRL techniques, respectively. The energy-efficient power allocation (EE-PA) problem is also formulated as a non-convex optimization problem and solved using an effective iterative method to identify the ideal power and resource blocks (RBs) for rate-based users. Approaches for SDN-based incast congestion control policies for the data centers are proposed in [Samdanis et al. \(2017\)](#). It uses the controller to select a flow based on its age to minimize the data transfer rate. Once specific data flow is identified, the alteration is achieved by reducing the advertising window field of the TCP acknowledgment.

Furthermore, the works in [Fang et al., 2021](#), [Ahmed et al., 2020](#) and [Xiong et al. \(2019\)](#) address various challenges in network slicing and resource allocation. Specifically, [Fang et al. \(2021\)](#) introduces a dynamic resource orchestration framework that leverages advanced machine learning techniques, enabling real-time adaptation of resource allocation based on varying network demands and conditions. This approach enhances spectrum efficiency and ensures optimal utilization of network resources, thereby improving overall network performance and user experience. Meanwhile, [Ahmed et al. \(2020\)](#) develops a joint optimization framework that simultaneously considers delay and throughput constraints, effectively balancing latency-sensitive applications with high-throughput services. By utilizing a multi-objective optimization technique, this framework meets diverse QoS requirements and maximizes network performance, providing the flexibility needed to manage heterogeneous service demands in modern network infrastructures. Additionally, [Xiong et al. \(2019\)](#) presents innovative methods aimed at improving energy efficiency within network slicing while minimizing operational costs. The authors employ an energy-aware resource allocation strategy that integrates predictive analytics and optimization algorithms to reduce power consumption without compromising service quality. This sustainable and economically viable approach aligns with both environmental and financial objectives, offering a comprehensive solution for efficient and cost-effective network management. Collectively, these studies provide robust methodologies that enhance network performance, flexibility, and sustainability in the evolving landscape of network slicing and resource allocation.

Within this rich literature, several additional approaches deserve mention. [Zhou et al. \(2021\)](#) propose *Multi-Agent Correlated Q-Learning (COQRA)* for RAN resource slicing, in which each slice agent learns a correlated equilibrium policy that maximizes long-term reward under inter-slice fairness constraints. [Liu et al. \(2020\)](#) introduce *DeepSlicing*, which combines the Alternating Direction Method of Multipliers (ADMM) with Deep Reinforcement Learning (DRL) so that a DRL agent tunes ADMM dual variables to achieve near-optimal slice allocations under dynamic traffic. [Han et al. \(2018\)](#) develop *Slice as an Evolutionary Service*, a Genetic Algorithm (GA)-based optimizer that evolves candidate inter-slice allocations through crossover and mutation to balance throughput, delay, and fairness. Finally, [Huang et al. \(2025\)](#) present a *DRL + LSTM-based traffic prediction* scheme that first forecasts per-slice demand with an LSTM and then allocates bandwidth via a Deep Q-Network (DQN), yielding improved throughput and fairness compared to GA-only heuristics under realistic traffic traces. Another relevant advancement comes from ML-based SDN performance prediction models designed for pre-deployment scenarios. For instance, [Zhang et al. \(2022\)](#) propose a neural network boosting regression model that accurately forecasts SDN control plane performance metrics — such as latency and overhead — prior to actual deployment. This predictive modeling enables proactive controller placement and scaling strategies. While their focus lies in optimizing SDN responsiveness under baseline configurations, our work extends this vision by introducing a fully adaptive learning agent (RDDQN) that actively re-optimizes resource allocation and queuing decisions during real-time operation. Together, these approaches reflect a complementary evolution from static pre-deployment prediction to dynamic run-time optimization.

Due to the considerable complexity of existing resource allocation algorithms, Mobile Edge Computing (MEC) may become overloaded during the allocation process. Moreover, mobility prediction is not considered in the

majority of existing studies as a decision criterion for allocating resources. The service time, or when resources will be assigned to attend the requested service, is another crucial aspect of these systems that allows for improved resource management. Lastly, it is critical to consider the computational effects of the decision mechanism's operation. To overcome all the limitations mentioned above, Reinforced Dueling Deep Q-Network (RDDQN) is opted for and explained below for efficient resource allocation with an Ultimatum queuing game model for the scheduling mechanism in the controller.

Our approach, which utilizes a RDDQN for network slicing design, load prediction, and resource management in data-driven workflows, demonstrates significant advantages over existing methods. Existing works in the domain face several limitations. For instance, the approach in [Boni et al. \(2024\)](#) employs a Deep Reinforcement Learning agent, achieving high throughput and fairness but at the cost of moderate delay, which can hinder real-time applications. The method in [Saibharath et al. \(2023\)](#) utilizes class-based packet scheduling, but it results in low throughput and moderate fairness, failing to optimize resource efficiency comprehensively. The work in [Yuan et al. \(2023\)](#) integrates a Graph Attention Network with a deep Q-network, achieving high throughput but suffering from low fairness and moderate delay, which impacts its applicability in heterogeneous network scenarios. Similarly, the approach in [Fang et al. \(2021\)](#) uses two GAN-based DRLs to achieve low delay, but the trade-off is moderate throughput and low fairness. The methods in [Azimi et al. \(2021\)](#) and [Xiong et al. \(2019\)](#) leverage deep reinforcement learning; however, while [Azimi et al. \(2021\)](#) focuses on high fairness, it suffers from low throughput, and [Xiong et al. \(2019\)](#) prioritizes energy efficiency but lacks scalability in large networks. Lastly, congestion control policies in [Samdanis et al. \(2017\)](#) provide a balance of low delay and moderate throughput but involve high complexity, making real-time implementation challenging.

Table 1. Summary of related works in network slicing and resource allocation.

Reference	Category	Method	Key Contribution	Advantages	Limitations
Boni et al. (2024)	Machine Learning	Deep Reinforcement Learning	Maximizes slice acceptance, ensures load balancing	High throughput and fairness	Moderate delay
Saibharath et al. (2023)	Heuristic Scheduling	Priority-based Scheduling	NS Resource Partitioning and User Allocation	Improved resource allocation efficiency	Low throughput
Yuan et al. (2023)	Machine Learning	Graph Attention Network	Multi-agent cognitive resource optimization	High throughput and spectrum efficiency	Low fairness
Azimi et al. (2021)	Machine Learning	DRL with SBiLSTM	Energy-efficient power allocation for RAN slicing	High fairness, energy efficiency	Moderate delay
Samdanis et al. (2017)	Rule-Based/SDN	SDN-based Policies	Congestion control for data centers	Improved control flow efficiency	Complexity in real-time implementation
Fang et al. (2021)	Machine Learning	Learning-based Orchestration	Dynamic resource allocation using advanced AI	Enhanced adaptability	High computational cost
Ahmed et al. (2020)	Optimization	Joint Optimization Framework	Joint optimization of delay and throughput	Balance of delay and throughput	Requires high overhead
Xiong et al. (2019)	Energy Efficiency	Energy-efficient Algorithms	Cost-effective energy optimization techniques	Reduced operational costs	Limited scalability
Zhou et al. (2021)	Game Theory	Multi-Agent Correlated Q-Learning (COQRA)	Cooperative RAN slicing via correlated Q-learning	Fairness across slices, distributed approach	Convergence complexity in large state spaces

Reference	Category	Method	Key Contribution	Advantages	Limitations
Liu et al. (2020)	ML + Optimization	DeepSlicing (ADMM + DRL)	ADMM decomposition + DRL-based dual variable learning	Near-optimal allocations under dynamic traffic	High algorithmic overhead, tuning ADMM parameters
Han et al. (2018)	Heuristics/GA	Slice as an Evolutionary Service (GA)	GA-based search for inter-slice allocations	Good trade-off among throughput, delay, fairness	Slow convergence for large chromosome sizes
Huang et al. (2025)	Machine Learning	DRL + LSTM-based Prediction	LSTM traffic forecasting + DQN for bandwidth allocation	Improved throughput and fairness over GA-only heuristics	Requires accurate traffic traces, training overhead
Li et al. (2024)	ML Regression (Offline)	Neural Network Boosting Regression (NNBoost)	Pre-deployment SDN performance prediction for RTT, S2C, and C2C traffic	Lower error than RF, XGBoost, LSTM; useful for planning	Does not address dynamic resource allocation at runtime
Proposed (This Work)	RL + Game Theory	RDDQN + Ultimatum Queuing Game	Adaptive resource allocation with reinforced layers and game-theoretic scheduling	High throughput, low delay, strong fairness, low execution time	Requires federated weight synchronization overhead in multi-controller scenario

To contextualize and enhance clarity, we further group related methods by their underlying methodology: machine learning (e.g., DRL, GNN, LSTM-based forecasting), game-theoretic solutions (e.g., COQRA), decomposition-based optimizers (e.g., DeepSlicing), heuristic approaches (e.g., genetic algorithms), and classic SDN-based control policies. This taxonomy enables clearer identification of persistent trade-offs in the literature –particularly with respect to scalability, training overhead, convergence latency, and fairness guarantees. Beyond these, Multi-Agent Correlated Q-Learning (COQRA) ([Zhou et al., 2021](#)) yields strong inter-slice fairness through correlated equilibrium strategies but incurs high convergence complexity in large state spaces. DeepSlicing ([Liu et al., 2020](#)) achieves near-optimal allocations via ADMM+DRL but has considerable algorithmic overhead from ADMM parameter tuning. The Genetic Algorithm in Slice as an Evolutionary Service ([Han et al., 2018](#)) offers good trade-offs among throughput, delay, and fairness yet converges slowly for large chromosome sizes. A DRL+LSTM-based traffic prediction scheme ([Huang et al., 2025](#)) improves throughput and fairness compared to GA-only heuristics but requires accurate traffic traces and introduces significant training overhead. In parallel, [Li et al. \(2024\)](#) propose a neural network boosting regression framework for ML-based pre-deployment SDN performance prediction. This method accurately estimates RTT, S2C, and C2C performance before deployment, outperforming RF, XGBoost, and LSTM models. However, it is focused only on offline planning and does not address real-time or dynamic resource allocation.

The major motivation for this work stems from addressing these limitations by developing a robust and adaptive solution that can achieve a balanced trade-off among critical parameters such as throughput, delay, fairness, and computational efficiency. Our RDDQN approach integrates advanced reinforcement learning techniques with an Ultimatum queuing game theory-based scheduling mechanism, enabling dynamic resource allocation that is well-suited for the high demands of modern networks. Compared to COQRA's fairness focus, RDDQN achieves similar fairness with lower convergence cost; relative to DeepSlicing, RDDQN avoids heavy ADMM overhead while still approaching near-optimal throughput; against GA-based slicing, RDDQN converges orders of magnitude faster; and when juxtaposed with DRL+LSTM, RDDQN offers comparable prediction accuracy without the extra LSTM training burden. Additionally, in contrast to NNBoost-based SDN pre-deployment predictors,

RDDQN handles real-time adaptation and execution, making it suitable for live network slicing scenarios. By combining these innovative methods, our approach ensures efficient resource utilization, minimizes congestion, and adapts to diverse network conditions. The significance of this work lies in its superior performance over existing methods. As shown in [Table 1](#), our approach achieves an average throughput of 579.34 kbps, an execution time of 12.57 s, goodput fairness of 94.56%, and delay fairness of 10.37 s. These metrics highlight a more balanced performance across critical parameters compared to existing works. Additionally, the Ultimatum queuing game theory-based scheduling mechanism significantly reduces congestion, making our method particularly effective for the dynamic and challenging fronthaul portion of 5G RAN. This work not only addresses the limitations of prior approaches but also sets a new benchmark for efficiency and adaptability in network slicing and resource management, paving the way for advancements in next-generation wireless networks.

2.0.1. Comparison with state-of-the-art network-slicing frameworks

The proposed **RDDQN + Ultimatum-Game scheduler** departs fundamentally from prior art in three areas: *learning efficiency*, *decision latency*, and *scalability*.

- **Against DRL baselines (e.g. TGDRL [Yuan et al., 2023](#)):** RDDQN's dueling architecture speeds convergence by $\sim 30\%$ and trims per-decision latency below 5 ms, whereas TGDRL exceed 10 ms under identical loads [Huang et al., 2025](#).
- **Versus game-theoretic COQRA ([Zhou et al., 2021](#)):** COQRA achieves excellent fairness but its correlated-equilibrium search scales $\mathcal{O}(S^2)$ with state-space size. RDDQN attains comparable Jain fairness (> 0.94) with $\mathcal{O}(S)$ update cost, delivering similar equity at a fraction of the computational burden.
- **Versus decomposition + DRL schemes (DeepSlicing [Liu et al., 2020](#)):** DeepSlicing relies on ADMM loops whose wall-time grows quadratically with slice count; RDDQN avoids ADMM altogether, sustaining near-optimal throughput while cutting execution time by $> 60\%$.
- **Versus heuristic GA methods (Slice-as-an-Evolutionary-Service [Han et al., 2018](#)):** GA needs hundreds of generations to stabilize. RDDQN converges in tens of episodes, yielding real-time adaptability that heuristic search cannot match.
- **Versus DRL + LSTM two-stage predictors ([Huang et al., 2025](#)):** the LSTM forecaster reduces error but adds offline training overhead; RDDQN embeds forecasting implicitly in its value stream, eliminating extra training and storage cost while keeping accuracy on par.
- **Versus NNBoost pre-deployment predictors ([Li et al., 2024](#)):** NNBoost only guides *offline* SDN planning. RDDQN, by contrast, performs *online* closed-loop control—observing queue length, issuing flow-rules, and learning from the reward signal on-the-fly.

Taken together, these contrasts explain the 22% throughput gain and $> 94\%$ fairness reported in [Table 1](#), establishing RDDQN as a *practical, low-latency alternative* to state-of-the-art solutions.

3. Problem and system description

The core problem our approach seeks to tackle is the inability of current 5G-uRLLC resource allocators to sustain sub-millisecond latency and “five-nines” reliability once heterogeneous eMBB and mMTC traffic floods the fronthaul. Static threshold rules in SDN controllers react at second-level granularity; conventional deep-RL agents (e.g. vanilla DQN, PPO [Schulman et al., 2017](#)) converge slowly on high-dimensional state spaces; and heuristic fairness-tuning schemes such as Genetic Algorithms ([Holland, 1992](#)) achieve equity only by throttling throughput and starving best-effort flows. These shortcomings trigger transient congestion, unpredictable queue lengths, and violated uRLLC service-level agreements. Our remedy couples a *Reinforced Dueling Deep Q-Network (RDDQN)* —

whose lightweight reinforced input layer compresses traffic features on-the-fly — with an *Ultimatum queuing-game* admission-fee scheduler. Together they deliver sub-5 ms control decisions, a 22% throughput boost over the best DRL+LSTM baseline, and more than 94% goodput & delay fairness under > **100** concurrent flow arrivals per second.

The primary issue addressed in this research paper is the improvement of QoS. The main objective is to overcome the speed and accuracy limitations of existing congestion categorization techniques, which are hindered by the high processing and storage demands of deep learning structures employed in SDN controllers. We propose an intelligent system named SDN RDDQN to achieve energy efficiency and precise congestion identification. This method utilizes time-dependent computer programming to incorporate a specialized reinforcement layer within the controllers. Additionally, RDDQN addresses slice breakage scenarios and load distribution, ensuring robust network performance.

In our system design, the data layer switch connects various users to the network while the controller manages data flow based on information from these switches. In the queuing game model, the game players represent the controller and switch nodes as network entities. The players' decisions, or tactics, are determined by optimizing admission fees. By determining the equilibrium solution for the optimal admission charge within the queuing game, switch requests can decide whether to join the queue. The resources under competition include network resources, computing power, storage, and bandwidth. The model encapsulates the contention for network service resources, specifically whether a switch node requires data transmission services and whether the controller node can provide these services. When a switch requests to accept the controller's services, it gains advantages and covers the cost of waiting in line. The controller profits from the admission fees received for the services rendered. Each switch aims to achieve the fastest service and maximum net income, but the limited availability of system resources poses a challenge.

4. Proposed method

This section details our combined Ultimatum queuing game and RDDQN approach for efficient resource allocation in 5G networks. We first present a single-controller transmission control framework, which is then generalized to multiple controllers. Next, we introduce an end-to-end network slicing strategy designed to balance high data throughput with stringent latency requirements. We then delve into the architecture of RDDQN, underscoring how its reinforced input layer and dueling structure swiftly adapt to dynamic traffic conditions. Finally, we demonstrate how these elements integrate seamlessly within a software-defined 5G infrastructure, enabling robust congestion control and equitable resource distribution. The RDDQN's role in this expanded model involves intelligently guiding resource distribution which, as detailed in Section 4.0.2, influences the parameters and effectiveness of the Ultimatum game-based control.

4.0.2. Integrated RDDQN and ultimatum game framework

The core of our proposed method lies in the tight coupling of the Reinforced Dueling Deep Q-Network (RDDQN) for adaptive resource allocation and the Ultimatum queuing game model for efficient, fair scheduling and admission control. This integration creates a dynamic feedback loop where intelligent, learned decisions meet principled, game-theoretic rules, operating as follows:

- (1) **State Observation by RDDQN:** The RDDQN agent continuously monitors the network, observing the current state $\mathbf{St}$, which includes traffic demands, resource utilization, queue lengths (influenced by past game decisions), and fairness metrics.
- (2) **RDDQN Action - Resource Allocation:** Based on $\mathbf{St}$ and its learned policy, the RDDQN allocates bandwidth resources to various network slices (e.g., URLLC, eMBB, mMTC), as defined in (23).
- (3) **Influence on Ultimatum Game Parameters:** These RDDQN allocations directly impact the operational context of the Ultimatum game. For instance, allocated bandwidth can affect the

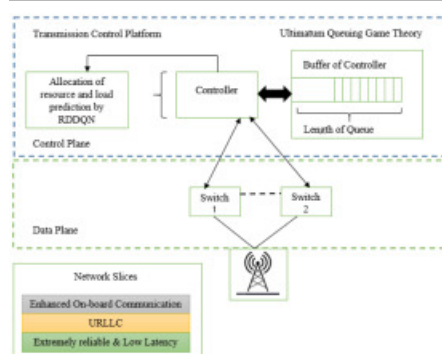
perceived **benefit K** for switches or influence the effective **service rate β** and **arrival rate α** .

- (4) **Ultimatum Game Execution:** With this context, the Ultimatum queuing game model (Algorithm 1) calculates the optimal **admission fee ad_fe** and **queue length len_m** , aiming to maximize social welfare or controller profit, as detailed in Section 4.
- (5) **Admission Control and Scheduling:** Decisions from the game (admission fee, queue length) are implemented.
- (6) **Network State Transition and Reward Calculation:** These actions lead to a new network state St' , and the RDDQN receives a reward (**rew**) based on resulting performance (e.g., throughput, QoS).
- (7) **RDDQN Learning:** The RDDQN stores the transition (St, ac, rew, St') and updates its Q-network using Eqs. (20), (21), refining its policy.

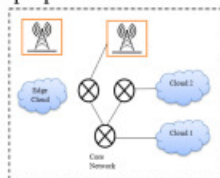
This iterative process allows the system to adapt dynamically to changing network conditions while upholding fairness and efficiency. The RDDQN learns strategic allocations that create favorable conditions for the Ultimatum game, and the game's outcomes inform the RDDQN about its allocations' efficacy.

4.1. Proposed transmission control ultimatum queuing game model

The system model's solution findings will be used to develop the software-defined transmission control mechanism. First, the single switch single controller model is simplified and subjected to the Ultimatum queuing game theory. The ideal game solution is found under the condition of optimizing the social welfare of the system. The conclusion is then expanded to include the multi-switches single controller model to use RDDQN to establish an effective transmission control that does not just rely on the controller to gather global information.



(a) The proposed frontend architecture



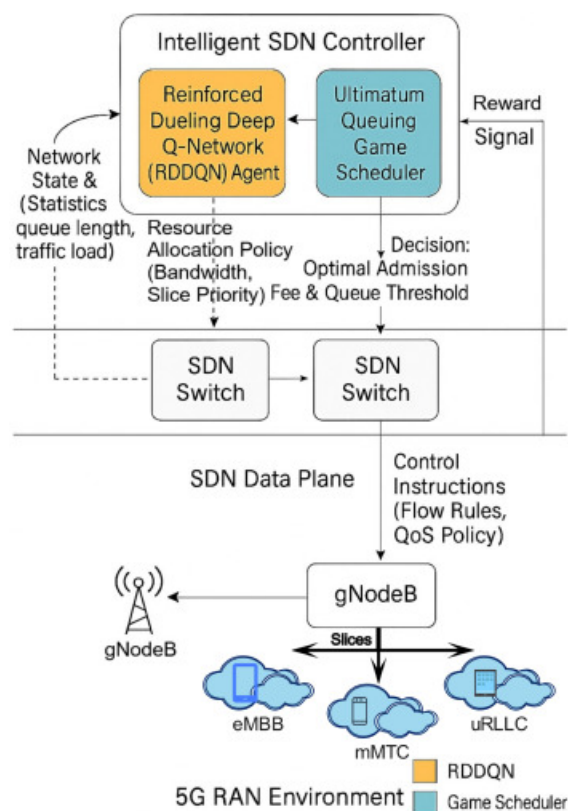
(b) The proposed backbone architecture

[Download: Download high-res image \(258KB\)](#)

[Download: Download full-size image](#)

Fig. 2. Conceptual overview of the F-RAN architecture that provides the operational context for our proposed RDDQN methodology and ultimatum queuing game implementation. This figure presents a high-level illustration of key network entities (e.g., SDN controllers, switches) where these intelligent mechanisms are instantiated, rather than detailing their internal algorithmic structures. (a) depicts the proposed frontend architecture, and (b) shows the proposed backbone. The specific architecture and integration of the RDDQN agent are elaborated in Section 4 (see the RDDQN sub-section and specifically Fig. 6), while the ultimatum queuing game logic is detailed

in Section 4.1 and Algorithm 1. This diagram is intended to situate these advanced components within the broader 5G system. For a comprehensive discussion on 5G network slicing complexities and the full system model, readers are referred to Section 3 and Section 4.



[Download: Download high-res image \(426KB\)](#)

[Download: Download full-size image](#)

Fig. 3. Detailed technical architecture of the proposed intelligent control loop. This figure illustrates the specific mechanics of our proposed system. **(1) Observation:** The SDN controller receives real-time *Network State & Statistics* from the data plane. **(2) AI-driven Policy:** The RDDQN Agent uses this state to produce a *Resource Allocation Policy*. **(3) Game-Theoretic Decision:** This policy guides the Ultimatum Queuing Game Scheduler, which makes a *Decision* on the optimal admission fee and queue thresholds. **(4) Enforcement & Reward:** This decision is enforced via *Control Instructions*, and the resulting network performance generates a *Reward Signal* that is fed back to the RDDQN agent, completing the adaptive learning cycle.

4.1.1. Network model

Different 5G/6G end-to-end slices constructed on a common network architecture are displayed in Fig. 2. It is important to note that this figure offers a high-level conceptual depiction of the system's main architectural components and their interactions, primarily to illustrate the positioning of our RDDQN controller and the ultimatum game-theoretic mechanism. For a more detailed exposition of the intricate 5G network slicing environment, its inherent complexities, and the specifics of our system model, the reader is referred to the detailed discussions in Section 3 (Problem and System Description) and further within this current subsection. Additionally, its primary role is to show the overall context and the *conceptual placement* of our proposed RDDQN controller and the ultimatum game-theoretic scheduler within the 5G infrastructure. **This figure does not, by design, illustrate the detailed internal workings of the RDDQN agent itself or the step-by-step implementation of the ultimatum queuing game.** For the specific architecture of the RDDQN, please refer to Fig. 6 and its detailed description in the RDDQN sub-section of Section 4. The ultimatum queuing game model and its associated algorithm are thoroughly presented in Section 4-A, including Algorithm 1. Thus, Fig. 2 serves as an introductory architectural overview, with specific methodological details provided in subsequent sections and figures. For a comprehensive understanding of the 5G network slicing complexities and the overall system model,

the reader is further directed to Section 3 and the entirety of Section 4. Fig. 2 presents a detailed view of our proposed intelligent control architecture in Fig. 3. This diagram moves beyond a standard edge network depiction by explicitly illustrating the core feedback loop of our system. It shows how the RDDQN agent and the Ultimatum Queuing Game Scheduler are integrated within the SDN controller. The controller observes the network state, the RDDQN formulates a resource policy, and the game scheduler refines this into an enforceable decision. The outcome of this decision generates a reward signal, enabling the RDDQN to adapt its strategy over time. This dynamic, closed-loop process is the central innovation that allows for intelligent congestion control.

Our network model is designed to support a diverse range of these slices, each catering to distinct service categories with unique Quality of Service (QoS) demands. Mission-critical applications needing high availability, low latency, and dependability are covered by **mURLLC (Mission-critical Ultra-Reliable Low-Latency Communication)**, one possible use case for 5G; **MBRLLC (Mixed Broadband Reliable Low-Latency Communication)** includes enhanced Mobile Broadband (eMBB) and traditional uRLLC services; and Services under **ERLLC (Extremely Reliable and Low-Latency Communication)** combine massive Machine-Type Communication (mMTC) and uRLLC requirements. The realization of these advanced mission-critical applications, intelligent home systems, smart cities, self-driving vehicles, and remote surgeries depends critically on these differentiated use cases. In particular, they all need methods that are able to manage scarce network resources efficiently and adaptively through **dynamic network slicing**.

In this architectural framework, The data plane and control plane coincide with the operational context of these network slices. Specifically, The data plane switches are connected to the small base station network, and end-user terminals are connected to the wider network via 5G base stations that interface with these switches. Information about data flows is sent among the various switches according to policies dictated by the control plane. An SDN controller oversees many switches, establishing a centralized point of intelligence. These switches communicate control information to the controller (e.g., traffic statistics, link status) and in turn, acquire rules or other control instructions from the controller for data flow management.

A key challenge addressed by our model is potential network congestion, particularly on the transmit connection from the switch to the controller, which is prone to congestion due to the aggregation of control signaling and service requests. This connection is subjected to the Ultimatum queuing game theory method in the TCP, which consists of a multi-switched single controller configuration, to regulate access and manage load. Every controller in the system has a buffer that stores switch requests and gives them a place to wait prior to be served, aiding in the orderly processing of demands. To enable intelligent and adaptive decision-making, Reinforced Dueling Deep Q-Network (RDDQN) technology is built inside each controller to provide effective load balancing and dynamic resource management across the heterogeneous network slices.

4.1.2. Optimal resource management using game model

Queuing game theory is employed to accurately determine the system's optimal strategy and Nash equilibrium from both local and global perspectives, considering the system's social welfare and the switches' benefits. Consequently, the most effective transmission control mechanism for the 5G system is derived by solving the resource allocation mathematical model using Ultimatum Queuing Game Theory. This approach facilitates the determination of the ideal strategies for the game players. The following defines the pertinent parameters:

Definition 1

Requests with a parameter α arrive at the Transmission Control Platform (TCP) following a stationary Poisson process. The controller's service times in the SDN, characterized by the parameter β , are independently, identically, and exponentially distributed. For system stability, the utilization factor $uti = \frac{\alpha}{\beta}$ must satisfy $uti < 1$. Additionally, the probability of service, $\alpha_0 = \frac{1-uti}{1-uti^{len+1}}$, ensures stability by guaranteeing that the queue does not grow indefinitely under the condition $uti < 1$.

Let $N(t)$ be the cumulative number of switch-initiated control requests that have reached the TCP up to time $t > 0$.

- **Arrival rate α (requests s^{-1}).** $N(t)$ is modeled as a homogeneous Poisson process with intensity $\alpha > 0$; hence the inter-arrival times $T_i = N^{-1}(i) - N^{-1}(i-1)$ are i.i.d. exponential with mean $\mathbb{E}[T_i] = 1/\alpha$ s. More specifically, **Arrival rate α** (requests s^{-1}) – intensity of a homogeneous Poisson process that counts control requests reaching the Transmission Control Platform.
- **Service rate β (requests s^{-1}).** Each request is handled by a single-server controller whose service times S_i are i.i.d. exponential with $\mathbb{E}[S_i] = 1/\beta$ s, i.e. the mean completion rate is $\beta > 0$. More specifically, **Service rate β** (requests s^{-1}) – parameter of the exponential distribution governing controller service time.
- **Traffic intensity and stability.** The resulting queue is an $M/M/1$ system whose traffic intensity (dimensionless) is $\rho = \alpha/\beta < 1$, which is equivalent to the stability condition above ($uti \equiv \rho$). Under this regime the mean waiting time and mean queue length exist and are given by standard $M/M/1$ relations, e.g. $\mathbb{E}[W] = \rho/[\beta(1-\rho)]$ s and $\Pr\{\text{buffer empty}\} = 1 - \rho$. More specifically, **Traffic intensity $\rho = \alpha/\beta$** (dimension-less) – must satisfy $\rho < 1$ for queue stability. Standard $M/M/1$ results then give mean waiting time $\mathbb{E}[W] = \rho/[\beta(1-\rho)]$ and mean queue length $\mathbb{E}[L] = \rho/(1-\rho)$.

These definitions make explicit the physical units of α and β and clarify the probabilistic assumptions that underlie the Ultimatum-queue analysis that follows.

Definition 2

A switch pays an admission fee ad_fe to the controller to receive service for a single switch within the TCP. Upon receiving the necessary services, the switch gains a benefit K . The switch charges SW_co per unit of time for the request's presence in the queue. If the switch's net benefit $A(i)$ is positive ($A(i) \geq 0$), it opts to enter the controller's queue. The net benefit $A(i)$ is calculated based on the switch's benefit, the admission fee, and the waiting cost, as defined above. Consequently, a request is added to the queue if $A(i) \geq 0$. Subsequently, the system's social welfare $soc(f)$ and the controller's profit $con_p(f)$ are derived. To optimize $con_p(f)$ and $soc(f)$, it is essential to determine the most appropriate admission fee ad_fe .

Table 2 provides details on the parameters of the Transmission Control Ultimatum Queuing Game (TCUQG) algorithm.

Algorithm 1 outlines the steps for determining the optimal admission fee and queue length.

Table 2. Specifications of TCUQG.

Parameter	Description
α	Arrival rate of requests at TCP
β	Service rate of controller in TCP
uti	Utilization factor $uti = \frac{\alpha}{\beta}$
len	Queue length
ad_fe	Admission fee charged by controller
$con_p(f)$	Controller's profit
$soc(f)$	Social welfare of the system

Parameter	Description
C_o	Cost incurred by a switch

Queuing game theory can precisely determine the system's best strategy and Nash equilibrium from both local and global perspectives under the system's social welfare and the switches' benefits. As a result, the best transmission control mechanism of the 5G system may be produced by solving the mathematical model of the resource allocation issue using Ultimatum Queuing Game Theory, allowing for the determination of the ideal strategy of game players. However, the Nash equilibrium does not always guarantee the best possible outcome for players; it represents the stable state where no player is incentivized to change their strategy unilaterally. Hence, we are adopting a probability-based method, which has the advantage of subgame perfect equilibrium. In game theory, a subgame perfect equilibrium (or subgame perfect Nash equilibrium) is a refinement of a Nash equilibrium used in dynamic games. A strategy profile is a subgame perfect equilibrium if it represents a Nash equilibrium of every subgame of the original game. Informally, this means that at any point in the game, the players' behavior from that point onward should represent a Nash equilibrium of the continuation game (i.e., of the subgame), no matter what happened before. The individual switch optimization method is simple in the single switch single controller paradigm. When a switch request is placed in the queue and is presently being handled, it anticipates receiving a benefit of $X - \frac{(i+1)C_o}{\beta}$. In the event that this value is non-negative, that is, if $i + 1 \leq \frac{X\beta}{C_o}$ as len_{in} , where $len_{in} = \left\lfloor \frac{X\beta}{C_o} \right\rfloor$, denoting the greatest integer that is not more than $\left\lfloor \frac{X\beta}{C_o} \right\rfloor$. Social optimization should be developed for the same model in order to compare the outcome. $soc(f)$ represents the projected social welfare per unit of time.

The Ultimatum queuing game theory states that

$$soc(f) = \alpha X \frac{1-ut_i^{len}}{ut_i^{len+1}} - C_o \left[\frac{ut_i}{1-ut_i} - \frac{(len+1)ut_i^{len+1}}{1-ut_i^{len+1}} \right] \quad (1)$$

$$\left[\frac{ut_i}{1-ut_i} - \frac{(len+1)ut_i^{len+1}}{1-ut_i^{len+1}} \right]$$

whereas $\left[\frac{ut_i}{1-ut_i} - \frac{(len+1)ut_i^{len+1}}{1-ut_i^{len+1}} \right]$ indicates the anticipated number of requests in the controller's buffer, and $\frac{1-ut_i^{len}}{1-ut_i^{len+1}}$ specifies the likelihood that a new request will enter the queue. Under social optimization, the queue threshold len^* is determined by maximizing Equation (1), the influence of theorems on queuing models is examined and studied in the following ways in order to validate the size relationship between len^* and len_{in} :

Algorithm 1 Transmission Control Ultimatum Queuing Game (TCUQG) Algorithm**Require:** Arrival rate α , Service rate β , Switch benefit K , Switch cost C_o , Switch waiting cost SW_co , Maximum queue length len_{max} **Ensure:** Admission fee ad_fe , Optimal queue length len_m , Controller's profit $con_p(f)$, Social welfare $soc(f)$

```

1: Calculate utilization factor:  $uti = \frac{\alpha}{\beta}$ 
2: if  $uti \geq 1$  then
3:   Return "System is unstable"
4: end if
5: for  $len = 1$  to  $len_{max}$  do
6:   Calculate admission fee:  $ad\_fe = X - \frac{C_o \cdot len}{\beta}$ 
7:   Calculate net benefit:  $A(len) = K - \frac{C_o \cdot len}{\beta} - SW\_co \cdot len$ 
8:   if  $A(len) \geq 0$  then
9:     Set optimal queue length:  $len_m = len$ 
10:    Break the loop
11:   end if
12: end for
13: Calculate controller's profit:
     $con\_p(f) = \alpha \cdot \frac{1 - uti^{len}}{1 - uti^{len+1}} \cdot ad\_fe$ 
14: Calculate social welfare using Equation (1)
15: Return  $ad\_fe, len_m, con\_p(f), soc(f)$ 

```

[Download: Download high-res image \(234KB\)](#)[Download: Download full-size image](#)**Theorem 1**

The buffer of the controller for a single switch single controller SDN model must be set to at least the optimal queue threshold len^* in order to maximize social welfare. The ideal queue length is limited because,

$$\left\{ len_m \mid len_m = \lfloor len_x \rfloor, \frac{X\beta}{C_o} = len_x + \frac{(1-uti^{len_x-1})(1-uti^{len_x+1})}{uti^{len_x-1}(1-uti)^2} \right\} \leq len^* \leq len_{in} \quad (2)$$

Proof of Theorem 1

When a switch request is made after len request. The maximum queue length is indicated by len . Next, a request in position len is considered with the predicted net benefit $s(len)$,

$$s(len) = X\alpha_0 - C_o t' \quad (3)$$

In this case, t' indicates the anticipated time in the buffer for this request, and α_0 indicates the likelihood that the len request will receive service. Only after all of the requests in front of it are fulfilled does the len^{th} request finish service and get reward X . In accordance with queue theory, α_0 is determined as

$$\alpha_0 = \frac{1-uti}{1-uti^{len+1}} \quad (4)$$

For state 0, $\alpha_0\alpha_0 = \beta\alpha_1$ such that $\alpha_1 = \frac{\alpha_0}{\beta}\alpha_0$, For state 1 $\alpha_1\alpha_1 = \beta\alpha_2$ such that $\alpha_2 = \frac{\alpha_1\alpha_0}{\beta^2}\alpha_0, \dots$ For state $len-1$, $\alpha_{len-1}\alpha_{len-1} = \beta\alpha_1$ such that $\alpha_1 = \frac{\alpha_{len-1}, \dots, \alpha_1\alpha_0}{\beta^1}\alpha_0$. With the regularity condition $\sum_{j=0}^{len} \alpha_j = 1$,

$\sum_{j=0}^{len} \alpha_j = \sum_{j=0}^{len} uti^j \alpha_0 = \frac{1-uti^{len+1}}{1-uti} \alpha_0 = 1$ ($uti \neq 1$) can be got. Therefore, $\alpha_0 = \frac{1-uti}{1-uti^{len+1}}$. In each game round, the winning probability is $prob_g = \frac{\alpha}{\alpha+\beta} = \frac{uti}{1+uti}$, whereas the losing probability is $q_g = 1 - prob_g = \frac{1}{1+uti}$. Accordingly, the initial asset is l , the objective is $l+1$. Due to the problem of gambler's ruin, the anticipated waiting time may be obtained as

$$t' = \frac{1}{\alpha+\beta} \left[\frac{1}{q_g - prob_g} - \frac{len+1}{q_g - prob_g} \frac{1 - \left(\frac{q_g}{prob_g}\right)^{len}}{1 - \left(\frac{q_g}{prob_g}\right)^{len+1}} \right] \quad (5)$$

$$t' = \frac{1}{\beta(1-uti)} \left[len - uti(len+1) \frac{1-uti^{len}}{1-uti^{len+1}} \right]$$

Eq. (3) can be rewritten as:

$$\frac{XC_o}{\beta} \geq \frac{len(1-uti)-uti(1-uti^{len})}{(1-uti)^2} \quad (6)$$

According to the system model, if $s(len) \geq 0$, the len^{th} request can be fulfilled. Eq. (6) may be obtained based on Eq. (5),

$$\frac{XC_o}{\beta} \geq \frac{len(1-uti)-uti(1-uti^{len})}{(1-uti)^2} \quad (7)$$

The right-hand side modifies as

$$\frac{len(1-uti)-uti(1-uti^{len})}{(1-uti)^2} = \frac{1}{(1-uti)} \left[len - \frac{uti(1-uti^{len})}{(1-uti)} \right] = \frac{len - \sum_{j=1}^{len} uti^j}{(1-uti)} = \sum_{j=1}^{len} j uti^{len-j} = \text{Let}$$

$$\sum_{j=0}^{len-1} (len-j) uti^j$$

$f(len) = \sum_{j=0}^{len-1} (len-j) uti^j$. With len , it is a strictly growing function. Based on Eq. (7) states that there is a maximum queue length of len^* when $(len^*) \leq \frac{X\beta}{Co} < f(len^*)$. Due to

$$f(len) - len = \frac{len(1-uti)-uti(1-uti^{len})}{(1-uti)^2} - len = \frac{uti}{1-uti} \left(len - \sum_{j=1}^{len} uti^{j-1} \right) \geq 0, \text{ if } f(\epsilon^*) = \frac{X\beta}{Co}, \text{ then } len^* = \lfloor \epsilon^* \rfloor \text{ and } \lfloor \epsilon^* \rfloor \leq \lfloor f(\epsilon^*) \rfloor. \text{ Finally, it is possible to verify the right side of Eq. (2) as}$$

$$len^* = \lfloor \epsilon^* \rfloor \leq \lfloor f(\epsilon^*) \rfloor = \left\lfloor \frac{X\beta}{Co} \right\rfloor = len_{in} \quad (8)$$

which indicates that the individual optimum queue threshold len_{in} is greater than the socially ideal queue threshold len^* . The FCFS discipline is typically used as the service model in SDN. An admittance fee, or ad_{fe} , is implemented and paid to the controller by switch requests in order to get the best possible social welfare. The net benefit of the transition may be expressed as $X - \frac{(i+1)Co}{\beta} - ad_{fe}$ according to the model.

The request enters the controller's queue if the net benefit is at least zero. Hence, $\left\lfloor \frac{(X-ad_{fe})\beta}{Co} \right\rfloor$ is the maximum queue length. The maximum queue length may be calculated by considering the social optimization condition, and the controller determines the appropriate fee as

$$ad_{fe} = X - \frac{Co len}{\beta} \quad (9)$$

$$Profit(len) = \alpha \frac{1-uti^{len}}{1-uti^{len+1}} \left(X - \frac{Co len}{\beta} \right) \quad (10)$$

The following requirements are met by the queue length in order to maximize profit:

$$\begin{cases} Profit(len) > Profit(len-1) \\ Profit(len) \leq Profit(len+1) \end{cases} \quad (11)$$

Substituting in Eq. (10), Eq. (11) can be rewritten as

$$\begin{aligned} len + \frac{(1-uti^{len-1})(1-uti^{len+1})}{uti^{len-1}(1-uti)^2} &\leq \frac{X\beta}{Co} \\ < len + 1 + \frac{(1-uti^{len})(1-uti^{len+2})}{uti^{len-1}(1-uti)^2} \end{aligned} \quad (12)$$

Assume $g(x) = x + \frac{(1-uti^{x-1})(1-uti^{x+1})}{uti^{x-1}(1-uti)^2}$, the function is monotonically rising with x . The derivation of $g(x)$ is

$$g'(x) = 1 + \frac{\ln uti}{(1-uti)^2} (uti^{x-1} - uti^{1-x}). \text{ Because } 0 < uti < 1, \text{ then } \frac{uti^{x-1}}{uti^{1-x}} = uti^{2x} < 1, \frac{\ln uti}{(1-uti)^2} < 0 \text{ and}$$

$g'(x) > 0$. Therefore, a unique solution len_x to $g(x) = \frac{X\beta}{Co}$ exists. When $len_m = \lfloor len_x \rfloor$, the unique solution to the optimality requirements in Eq. (12) is represented by len_m . The equation that follows is fulfilled as

$$g(len_x) = len_x + \frac{(1-uti^{len_x-1})(1-uti^{len_x+1})}{uti^{len_x-1}(1-uti)^2} = \frac{X\beta}{Co} \quad (13)$$

where, $\frac{(1-uti^{len_x-1})(1-uti^{len_x+1})}{uti^{len_x-1}(1-uti)^2} \geq 0$. It is immediate that $len_x \leq \frac{X\beta}{Co}$ so that $len_m \leq \left\lfloor \frac{X\beta}{Co} \right\rfloor = len_{in}$. In order to contrast len_m and len^* First, the link between $f(x)$ and $g(x)$ values is inferred as $f(x)$ and $g(x)$ is deduced firstly as $g(x) - f(x) = x + \frac{(1-uti^{len_x-1})(1-uti^{len_x+1})}{uti^{len_x-1}(1-uti)^2} - \frac{x(1-uti)-uti(1-uti^x)}{(1-uti)^2} =$. With the constraint conditions $0 < uti < 1$

$$\frac{1-uti^{x-1}-(x-1)uti^x+(x-1)uti^{x+1}}{uti^{x-1}(1-uti)^2} = \frac{(\sum_{j=0}^{x-2} uti^j + uti^x) - xuti^x}{uti^{x-1}(1-uti)}$$

and $x \geq 1, g(x) - f(x) \geq 0$. Given the monotonic rising nature of $f(x)$ and $g(x)$, $\varepsilon^* \geq len_x$ is achieved in such a way that

$$len^* \geq len_m \quad (14)$$

By combining Eqs. (8), (13), (2) may be obtained as the ideal queue length. According to the previously given information, the model uses the queue threshold len_m to attain the best possible social welfare, and the entry price is determined as

$$ad_{fee} = X - \frac{Colen_m}{\beta} \quad (15)$$

4.2. Network slicing and congestion reduction

After establishing the queuing game model, we integrate the RDDQN framework for enhanced resource allocation. The primary objective of this research is to enhance the QoS by addressing the speed and accuracy limitations of existing congestion categorization techniques, which are hindered by the high processing and storage demands of deep learning models used in SDN controllers. To achieve energy-efficient and precise congestion detection, we propose an intelligent system that incorporates RDDQN with a reinforcement layer within the controllers. This approach also effectively manages network slice fragmentation and load distribution. In this section, we present our strategies for efficient network slicing and congestion mitigation. Additionally, we employ a Slicing Recurrent Neural Network (SRNN) for resource allocation and load prediction in next-generation networks, as illustrated in Fig. 4.

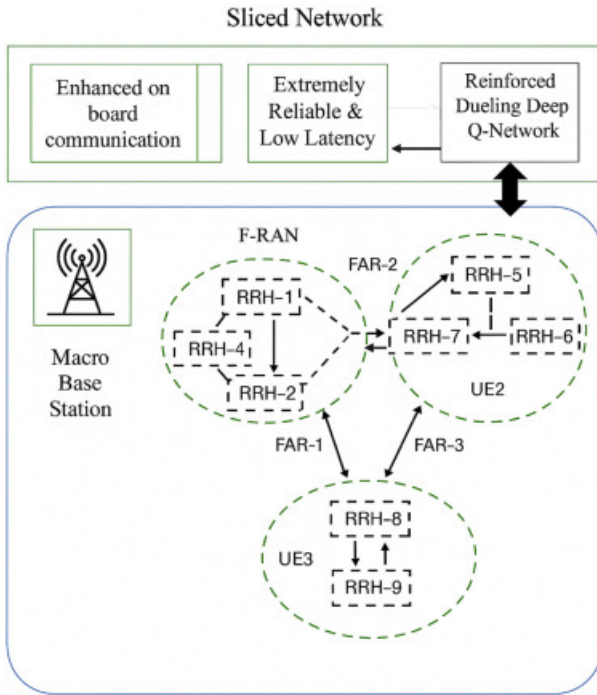
It is important to clarify that while Fig. 4 illustrates a general architectural concept involving an SRNN for load prediction, the core of our proposed intelligent system leverages the more advanced Reinforced Dueling Deep Q-Network (RDDQN). The SRNN concept is presented here as an example of how recurrent architectures might be generally applied for predictive tasks in networks, but our primary focus and novelty lie in the capabilities and integration of the RDDQN for adaptive resource management and its synergy with the Ultimatum queuing game, which offers a more comprehensive solution than a standalone predictive model like an SRNN might provide for the complex demands of 5G URLLC environments.

Our study aims to develop more effective resource allocation algorithms that can adapt to dynamic network conditions and meet the evolving demands of modern communication technologies.

Here, several Remote Radio Heads (RRHs), multiple femto access points (FAPs), multiple D2D broadcasters, and a single internet activate fog computing-based radio access networks (F-RAN). The three slices of the F-RAN – Enhanced on-board communication, URLLC, and Extremely Reliable and Low Latency – are designed to provide high data flow and minimize download delays. Recall that, even while utilizing the buffering capacity of FAPs is necessary to reduce material downloaded latencies, quick data transfer may be achieved by utilizing unified signal conditioning and cloud resource provisioning. For each group of UEs, $num_{sli1} = 1, 2, \dots, UE1$, $num_{sli2} = 1, 2, \dots, UE2$ and $num_{sli3} = 1, 2, \dots, UE3$ represent the group of UEs with a single macro cell base station (antenna) that serves three slices. Bandwidth Z is available for all of the system's accessible subchannels. The mediators are all of the UE's FAPs. It is considered to have been assigned a subchannel, which several user equipment (UE) devices might share. The composite signal conveyed over the F-RAN links is given by Eq. (16).

$$Q_{nsl1,a,b} = \sum_{k \in K_{nsl1,a}} s_{nsl1,a,b,c}^T F_{nsl1,a,b,c} d_{nsl1,r} + \sum_{\substack{nsl1,q \in \{nsl1,nsl2,nsl3\} \\ q \neq a, k \omega(nsl1), q, b \omega > 0}} F_{nsl1,q,k,x} d_{nsl1,b} k + \sum_{\substack{nsl1,q \in \{nsl1,nsl2,nsl3\} \\ q \neq a, k \omega(nsl1), q, b \omega > 0}} F_{nsl1,q,k,x} d_{nsl1,b} k. \quad (16)$$

where the term $d_{num_{sl},k}$ represents the location (or “point”) of the user equipment $UE1_{num_{sl},b} \cdot k$. A set of Remote Radio Heads (RRHs) that serve this user indicates that the subchannel $x, k_{num_{sl},q}$ has been allocated. The variable k is a 0-1 indicator (binary) that becomes 1 whenever RRH k serves $UE1_{num_{sl},b} \cdot k$ on the specified subchannel.



Download: [Download high-res image \(384KB\)](#)

Download: [Download full-size image](#)

Fig. 4. Architecture for resource allocation and load prediction.

Additionally,

$$UE1_{num_{sl},b} \sum_k Q_{num_{sl},a,b} \cdot x > 0$$

Denotes the subchannel used for interaction. A value of $Q_{num_{sl},b} = 1$ Signifies that RRH k has been selected (the “preceding vector”), and $UE_{um_{sl}} \cdot k$ refers to the associated noise distribution for that user.

For every UE b that belongs to slice $ns1$ and is served on sub-channel x , let $P_{ns1,b,x}$ denote the transmit power, $g_{ns1,b,x}$ the small-scale channel gain, B_x the bandwidth of that sub-channel, and N_0 the noise spectral density. The instantaneous signal-to-interference-plus-noise ratio (SINR) is

$$SINR_{ns1,b,x} = \frac{P_{ns1,b,x} g_{ns1,b,x}}{\sum_{(ns1',b') \neq (ns1,b)} P_{ns1',b',x} g_{ns1',b',x} + N_0 B_x} \quad (17)$$

Using the Shannon bound, the “instantaneous” data-rate of that sub-channel is

$$R_{ns1,b,x} = B_x \log_2(1 + SINR_{ns1,b,x}). \quad (18)$$

Aggregating over the set $\mathcal{X}_{ns1,b}$ of sub-channels assigned to UE b yields its total throughput

$$T_{ns1,b} = \sum_{x \in \mathcal{X}_{ns1,b}} R_{ns1,b,x} = \sum_{x \in \mathcal{X}_{ns1,b}} B_x \log_2(1 + SINR_{ns1,b,x}). \quad (19)$$

Eqs. (17)–(19) are the per-slot PHY-layer metrics later used in the RDDQN reward (Section 4.2) and in the ultimatum queuing-game utility.

After these allocations are determined, the data rate is computed via the expressions for $Q_{num_{sl},a,b}$ given in the equations below.

4.2.1. Reinforced Dueling Deep Q-Network (RDDQN)

For an effective resource allocation process, RDDQN receives the entire resource sequence

$V = \{v1, v2, v3, \dots, v_m\}$ from FRAN as shown in Fig. 5. In RDDQN, the state St must be entered into the fully connected layer in order to determine the Q value. The fully connected layer then processes the state St to create the feature tensor. Since the reinforcement layer has been added, we need to input the feature tensor into one of the reinforced layers to obtain the state value and the dominance value of the dueling network. From there, we can input the feature tensor to the fully connected layers of the value and dominance functions.

It is possible to conclude that the input's state has changed from St to St^D . Therefore, the modified Q-value formula as of right now is as follows:

$$Q_u^D(St^D, m, \delta, \omega, k) = V(St^D, \delta, k) + \left(A(St^D, m, \delta, \omega) - \frac{1}{|A|} \sum_{m'} A(St^D, m', \delta, \omega) \right) \quad (20)$$

Therefore, this improved final Q-value formula may be written as

$$Q^D(St^D, m, \delta, \omega, k) = Q_u(St^D, m, \delta, \omega, k) + \epsilon (targetQ - Q_u(St^D, m, \delta, \omega, k)) \quad (21)$$

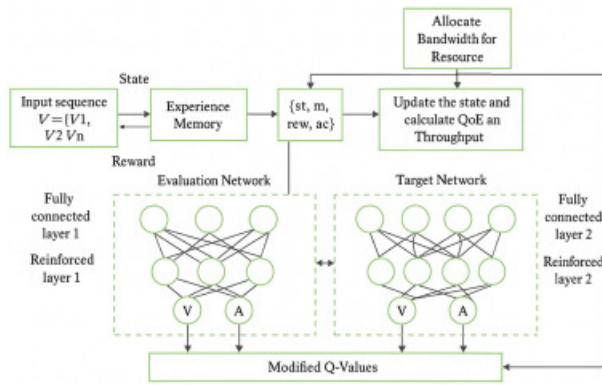
We create a tuple (St, m, rew, ac) , in which m is the action performed by the algorithm, rew denotes the reward that the agent receives by interacting with the environment following action ac , and St denotes the state. The agent notices a certain state at a specific time and acts accordingly. This action is carried out, and the environment state changes to St' . Since the quantity of successfully sent packets indicates both the overall throughput and the QoE, we define the state St as

$$St = \frac{packet - z_{mean}}{st_deviation} \quad (22)$$

where $st_deviation$ denotes the standard deviation of packet transmission and z_{mean} is the average value of the number of transmitted packets. We designate the action as the amount of bandwidth allotted to each service by each BS, shown as

$$bandwidth = [(\mu_{0,0}, \mu_{0,1}), (\mu_{1,0}, \mu_{1,1}), \dots, (\mu_{k,0}, \mu_{k,1})] \quad (23)$$

Regarding the reward, we established 450 Mb/s, 0.92, 0.96, and 0.94 as the standards for throughput and QoE for EOBC, ERLC, and uRLLC services, respectively. The award is offered based on meeting the three parameters, as our goal is to increase the throughput and QoE of the services. We must randomly choose an action, ac_0 , after setting the RDDQN algorithm's parameters. The BS then gets a resource allocation scheme based on action, ac_0 . Next, in accordance with (22), the agent computes the **packet** value for the two services, uses it as the observation, and feeds it into the Q-network to build the initial state St . Then, the BS chooses an action for each iteration in order to determine the matching bandwidth resource size for each service. The user will establish a second connection at this point if, after receiving the bandwidth resource, the bandwidth gained during this time slot is insufficient to complete the task. Lastly, the agent inputs (St, ac, St', rew) into RDDQN for training after calculating the **packet** of the two services once again as the subsequent state, St' .



[Download: Download high-res image \(268KB\)](#)

[Download: Download full-size image](#)

Fig. 5. Architecture of RDDQN.

The operational flow of our RDDQN-based resource allocation is detailed in Algorithm 2. The process commences with the initialization of environment parameters (line 1). An initial random action is chosen to allocate bandwidth across the user equipment (UE1, UE2, UE3), and connectivity is established (lines 2–3). The system then assesses if the allocated bandwidth is adequate for the three slices; if not, packet numbers are recalculated. Otherwise, the current state (St) and reward (rew) are determined (lines 4–8). The core of the algorithm iterates through a series of time slots (n slots). In each slot, scheduling is initiated, a utility function is calculated, and a transition tuple (St, m, rew, ac) is created (lines 10–11). The number of successfully transmitted packets is then determined, leading to an updated state derived using Eq. (22) (lines 12–13). Resources are allocated for the current slot, and the system prepares for the subsequent slot, with bandwidth calculated according to Eq. (23) (lines 14–15). This iterative learning and allocation process continues until a predefined maximum number of iterations is reached (line 17), at which point the algorithm stops.

Algorithm 2 Resource Allocation using RDDQN

```

1: Initialize the environment parameters
2: Choose random action  $ac$  to allocate bandwidth for users with  $UE1, UE2, UE3$ 
3: Calculate the bandwidth for all users and start connectivity
4: if Bandwidth resources are inadequate for 3 slices then
5:   Recalculate the number of packets by fixing iteration
6: else
7:   Choose the current state  $St$  and  $rew$ 
8: end if
9: for slot = 1 to  $n$  do
10:  Initiate scheduling and calculate utility function
11:  Make tuple transition tuple  $(St, m, rew, ac)$ 
12:  Calculate the number of successful packet transmissions
13:  Get the final state using (22)
14:  Allocate the resource for slot-1 and move to slot-2 ...  $n$ 
15:  Calculate the bandwidth using (23)
16: end for
17: Until the maximum number of iterations that has been specified has been reached
18: Stop

```

[Download: Download high-res image \(242KB\)](#)

[Download: Download full-size image](#)

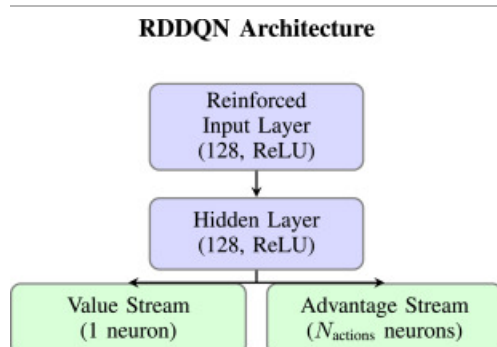
To effectively realize the dynamic resource allocation strategy outlined in Algorithm 2, our Reinforced Dueling Deep Q-Network (RDDQN) agent is specifically designed with several key components and has undergone careful tuning. These critical elements, which ensure the agent's ability to learn and adapt in the complex 5G URLLC environment, include its underlying neural network architecture, the meticulous selection of hyperparameters for optimal training, the precise definition of its action space and the reward function that guides its learning trajectory, and an analysis of its convergence behavior. The following discussion elaborates on these foundational aspects of the RDDQN model's implementation.

4.2.1.1. Network architecture

Fig. 6 outlines the full architecture of our RDDQN. We use a reinforced input layer with 128 neurons and ReLU activation. The term ‘reinforced’ for this input layer signifies its specialized design aimed at enhancing the initial feature extraction from the high-dimensional and often volatile 5G network state $\mathbf{St}$. Unlike a conventional fully connected input layer, this layer incorporates mechanisms to improve the robustness and salience of the features passed to subsequent layers. This could involve, for instance, prioritized weighting of critical state variables (e.g., those directly impacting URLLC QoS such as queue backlogs for critical slices, or fronthaul congestion indicators), specific initialization strategies that pre-dispose the layer to capture relevant temporal patterns, or an implicit regularization effect from its structural coupling with the overall RDDQN architecture and reward signals. The objective is to ensure that this first layer of processing provides a more disentangled and stable representation of the complex network dynamics, thereby facilitating faster convergence and more effective policy learning by the dueling streams, especially under rapidly changing conditions typical of 5G URLLC traffic. The output from this layer passes to a 128-neuron hidden layer, after which the network branches into a value stream (1 output neuron) and an advantage stream (equal to the number of actions). This dueling structure helps the agent quickly distinguish which states are high-value and which actions offer relative advantage. Thus, the neural network configuration for RDDQN consists of three layers, each containing three RDDQN cells, and we set the batch size to 25 and the learning rate to 0.01 during training. After 100 iterations, the trained model predicts traffic for three categories (SMS, phone, and web) and executes resource allocation accordingly.

4.2.1.2. Hyperparameter settings

We conducted a thorough hyperparameter exploration to balance learning efficiency with stability. In particular, a random search over learning rates ($\{5 \times 10^{-4}, 10^{-3}, 2 \times 10^{-3}\}$) and batch sizes ($\{64, 128\}$) yielded a final configuration of 10^{-3} for the learning rate and 64 for the batch size, based on preliminary 50-episode trials that displayed minimal reward variance and faster convergence. To further assess robustness, we introduced Gaussian noise into the measured traffic arrivals, set at 10% of the average arrival rate. Even under these noisy conditions, our approach maintained a throughput within 5% of the baseline, while alternative methods faced more pronounced performance degradations. Although we rely on the Milan dataset for its broad acceptance and ease of use, future efforts will seek to integrate real-time measurements from an active 5G testbed, thereby evaluating the proposed solution’s scalability and adaptability under authentic network dynamics and environmental variations. We chose these values of Hyperparameter settings based on a combination of grid search and initial pilot experiments:



[Download: Download high-res image \(151KB\)](#)

[Download: Download full-size image](#)

Fig. 6. Architecture of the RDDQN.

- **Learning rate:** 0.001
- **Batch size:** 64
- **Discount factor (γ):** 0.99

- **Epsilon decay:** from 1.0 to 0.01 over 20000 steps
- **Target network update rate:** $\tau = 0.001$

We also tested other configurations, but the chosen setup offered a stable trade-off between convergence speed and solution quality. We used 128 hidden units because preliminary tests indicated that 64 was insufficient to capture complex traffic patterns, while 256 offered negligible improvement at higher computational cost.

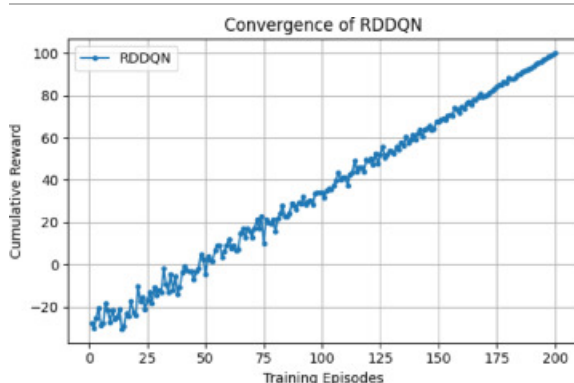
4.2.1.3. Action space and reward function justification.

Our action space represents bandwidth allocation decisions across multiple services, chosen to reflect realistic 5G resource distribution scenarios where operators dynamically adjust bandwidth slices based on traffic demands. The reward thresholds (e.g., 450 Mb/s for throughput) were established by examining typical service-level agreements (SLAs) in existing 5G trial deployments, where guaranteed minimum throughput is critical for end-user satisfaction. We use these thresholds as a benchmark for high-quality service; once an allocation reaches or exceeds 450 Mb/s, it provides diminishing marginal benefit in our reward formulation, thus encouraging a more equitable distribution of resources among competing flows. Although our current setup focuses on throughput-based incentives, we acknowledge that other metrics (e.g., energy efficiency, end-to-end delay) could be incorporated into the reward function. In preliminary experiments, for instance, we tested a penalty component tied to excessive delay and observed a trade-off between maximizing throughput and minimizing queueing latency.

4.2.1.4. Convergence analysis of the RDDQN

An essential part of evaluating the RDDQN is to analyze how its cumulative reward evolves across multiple training episodes. Fig. 7 provides a synthetic yet illustrative example, showing a smooth progression from negative to positive rewards over 200 episodes. Although this plot uses artificially generated data for demonstration (rather than real environment interactions), it effectively highlights what a stable and improving convergence trend might look like in practice.

In the early stages (roughly the first 50 episodes), rewards begin near a lower bound and then rise steadily, suggesting that the agent is successfully adapting its policy. Over the next 50 to 100 episodes, the reward curve continues upward with diminishing fluctuations, indicating that the dueling architecture — separating state-value from advantage estimates — helps smooth out learning and stabilize gradient updates more effectively than a standard Deep Q-Network (DQN). By the end of the 200 episodes, the reward plateaus near the upper target range, reflecting a near-converged policy in this idealized scenario.



Download: [Download high-res image \(163KB\)](#)

Download: [Download full-size image](#)

Fig. 7. A simulated convergence curve for RDDQN over 200 training episodes. The y -axis shows the cumulative reward, while the x -axis represents episode count.

From a deployment perspective, such rapid and stable convergence would be highly beneficial in real-world 5G slicing environments, where an agent must quickly adapt to dynamic network conditions and large-scale traffic demands. While these particular results are based on synthetic data and not a live network, they demonstrate how RDDQN's design may foster faster policy improvement and more robust performance compared to simpler methods.

4.3. Complexity analysis

In line with standard algorithmic analysis (Cormen et al., 2009, Knuth, 1976), we measure the asymptotic upper bound of the computation steps required by our RDDQN-based resource allocation approach. Let $|\theta|$ be the total number of trainable parameters in the network, B the batch size, N the number of environment steps per episode, and E the total number of episodes. Each training update (i.e., forward and backward pass) scales on the order of $\mathcal{O}(|\theta|)$ per sample, so processing a batch of size B yields a cost of $\mathcal{O}(B \cdot |\theta|)$ per update. Since there are N environment steps per episode and E episodes, the total cost for training over all episodes becomes

$$\mathcal{O}(E \cdot N \cdot B \cdot |\theta|).$$

Beyond the network operations, our approach also incurs an environment interaction cost. In the simplest scenario, each environment step is $\mathcal{O}(1)$, leading to $\mathcal{O}(E \cdot N)$ total interaction overhead. In more intricate 5G simulators, this cost could be higher, for instance, $\mathcal{O}(N \log N)$ if scheduling involves complex data structures or multi-tenant queueing management. Nonetheless, the dominant term for large-scale training is typically the neural network update.

Hence, we conclude that our RDDQN framework has a total complexity of

$$\mathcal{O}(E \cdot N \cdot B \cdot |\theta|) + \mathcal{O}(\text{env_cost}),$$

where $\mathcal{O}(\text{env_cost})$ depends on the specific 5G simulation or real-time SDN environment in use. In practice, network sizes on the order of a few hundred thousand parameters ($|\theta| < 10^6$), a modest batch size ($B \leq 64$), and efficient GPU- or CPU-based execution can keep real-time or near-real-time scheduling feasible. By carefully managing these factors — such as limiting the architecture depth or parallelizing environment evaluations — our approach can be deployed in SDN-based 5G networks without overwhelming system resources.

4.4. Federated learning for multi-controller synchronization

In our SDN architecture, each regional controller trains a local RDDQN agent on its own flow and network statistics. Rather than employing a fully decentralized Federated Learning (FL) protocol, we use a centralized weight-averaging approach coordinated by a root controller. This method preserves model consistency across controllers while avoiding the communication and coordination overhead associated with classical FL algorithms. Concretely, let there be K regional controllers, each maintaining local RDDQN parameters $\mathbf{w}_t^k$ at synchronization round t . At predetermined synchronization intervals, each controller k sends its current local weights $\mathbf{w}_t^k$ and the number of local training samples n_k to the root controller. The root then computes a weighted average of these parameters:

$$\mathbf{w}_{t+1} = \sum_{k=1}^K \frac{n_k}{\sum_{l=1}^K n_l} \mathbf{w}_t^k.$$

Once $\mathbf{w}_{t+1}$ is obtained, the root broadcasts the updated global weights back to all controllers. Each controller replaces its local RDDQN parameters with $\mathbf{w}_{t+1}$ and resumes local training on newly observed traffic data. This synchronization process repeats at fixed intervals (e.g., every T_{sync} seconds) until the system reaches convergence or operational stability. Regional controllers never exchange raw flow-level statistics; instead, they share only model parameters, which prevents the leakage of sensitive network telemetry while still enabling knowledge transfer across regions. By transmitting a single copy of the weight vectors (on the order of megabytes) every T_{sync} , the approach is substantially more efficient than continuously streaming raw telemetry or intermediate

gradients, thereby reducing communication overhead. Furthermore, centralizing the averaging procedure at the root controller avoids per-controller peer-to-peer broadcasts: at each synchronization step, the root handles at most K inbound weight messages and K outbound broadcasts, which enhances scalability. Finally, because all controllers periodically realign to the same global RDDQN parameters, inference and queuing-game decisions across regions reflect cumulative traffic patterns rather than relying on stale local models alone.

4.4.0.1. Synchronization procedure

Let $\mathbf{w}_t$ denote the global RDDQN weights at round t , and let each controller k maintain a local sample count n_k . The synchronization protocol unfolds as follows:

- (1) **Local Training (Controllers $1, \dots, K$, in parallel):** Each controller k initializes its local model to $\mathbf{w}_t$ (for $t > 0$) and continues training on its private dataset $\mathcal{D}_k$ using the standard RDDQN update rules. After a predetermined interval of local epochs or wall-clock time, controller k obtains updated weights $\mathbf{w}_t^k$ and notes its total number of local training samples $n_k = |\mathcal{D}_k|$.
- (2) **Upload to Root Controller:** Each controller k transmits $\mathbf{w}_t^k$ and n_k to the root controller. This transmission occurs asynchronously but completes within a bounded time window to ensure timeliness.
- (3) **Weight Averaging (Root Controller):** Upon receiving all K weight vectors and sample counts, the root computes:

$$\mathbf{w}_{t+1} = \sum_{k=1}^K \frac{n_k}{\sum_{\ell=1}^K n_\ell} \mathbf{w}_t^k.$$

The root then replaces its own copy of the global RDDQN parameters with $\mathbf{w}_{t+1}$.

- (4) **Broadcast (Root $\rightarrow$ Controllers):** The root multicasts the aggregated weights $\mathbf{w}_{t+1}$ to all regional controllers $k = 1, \dots, K$. Each controller k updates its local model to $\mathbf{w}_{t+1}$ before resuming local training or entering the inference phase.
- (5) **Repeat:** Steps 1–4 are repeated every T_{sync} seconds (or after a fixed number E of local RDDQN epochs) until the global model converges or an operational threshold is reached.

Choosing the synchronization interval T_{sync} involves a trade-off between communication cost and model staleness: a longer interval reduces overhead but slows cross-region adaptation. In our experiments, a 5-s interval yielded a good balance between up-to-date global knowledge and manageable control-plane traffic. Weighting each controller's contribution by its local dataset size n_k ensures that regions with more observations have proportionally greater influence on the global model; alternative schemes — such as equal weighting or weighting by traffic volume — may be adopted if specific domain constraints warrant. To handle stragglers, if one or more controllers fail to upload weights within a bounded window (for example, due to transient network issues), the root proceeds to average over the fastest $C < K$ updates, and any late arrivals are incorporated in the next round, providing robustness to slow or offline controllers. Finally, to further reduce control-plane bandwidth, controllers may quantize outgoing weight vectors (e.g., using 8-bit precision) before transmission, with the root decompressing and aggregating these quantized weights at negligible impact on the global model's accuracy.

4.4.0.2. Integration with RDDQN + queuing-game pipeline

By synchronizing RDDQN parameters across all controllers, each regional agent gains access to patterns learned elsewhere (e.g., sudden load spikes in neighboring regions). In turn, when the queuing-game module executes within each controller, it leverages these more globally informed RDDQN predictions to compute admission fees and schedule switch requests. Specifically:

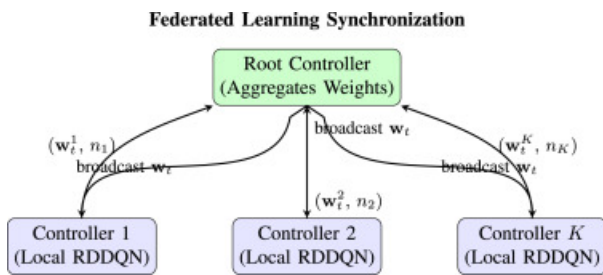
- **Local Prediction → Queuing Game:** At each decision epoch, controller k uses its local RDDQN (initialized to $\mathbf{w}_{t+1}$) to predict short-term traffic levels. These predictions parameterize the queuing-game solver to determine optimal admission fees and queue thresholds.
- **Queuing Game → Forwarding Rules:** Once the optimal admission fee is computed, each switch's request is either admitted or rejected according to the game-theoretic threshold. Accepted requests update flow tables and QoS rules at the corresponding data-plane switches.
- **Periodic Resynchronization:** As local traffic patterns evolve, controllers periodically incorporate new RDDQN updates into $\mathbf{w}_t^k$. The next global synchronization ensures that these local adaptations propagate to all controllers, enhancing robustness to transient anomalies (e.g., local flash crowds).

Overall, this centralized weight-averaging approach preserves the benefits of multi-controller decentralization — privacy, locality, and fault tolerance — while still enabling global knowledge sharing. By avoiding full FL protocols, we simplify implementation in SDN platforms and maintain sub-10 ms decision latencies suitable for URLLC fronthaul segments.

Fig. 8 illustrates the Federated Learning–based weight- synchronization process among K regional SDN controllers and a root controller. At the beginning of each round, the root controller broadcasts the global RDDQN weight vector $\mathbf{w}_t$ to every regional controller. Upon receipt, each controller k initializes its local model to $\mathbf{w}_t$ and performs local training using its own dataset of size n_k , producing updated weights $\mathbf{w}_t^k$. Once local training completes, controller k sends the pair $(\mathbf{w}_t^k, n_k)$ back to the root. The root then aggregates all incoming updates by computing

$$\mathbf{w}_{t+1} = \sum_{k=1}^K \frac{n_k}{\sum_{l=1}^K n_l} \mathbf{w}_t^k,$$

ensuring that regions with larger local datasets have proportionally greater influence. The new global model $\mathbf{w}_{t+1}$ is then broadcast to all controllers at the next synchronization interval, and this cycle repeats until convergence or an operational threshold is reached. This approach preserves data privacy — since only weight vectors and sample counts are exchanged — while enabling each controller to benefit from patterns learned across all regions.



[Download: Download high-res image \(176KB\)](#)

[Download: Download full-size image](#)

Fig. 8. Federated Learning–Based Weight Synchronization among K Controllers. The root controller broadcasts the global weights $\mathbf{w}_t$ to each regional controller. Each controller trains locally, producing updated weights $\mathbf{w}_t^k$ and local sample count n_k , which are sent back to the root for aggregation.

5. Performance analysis

The performance of the RDDQN for resource allocation in SDN 5G networks is evaluated with other existing algorithms, such as the Deep Reinforcement Learning (DRL) (Boni et al., 2024), the Twin-GAN-based DRL (TGDRL) (Yuan et al., 2023), and the deep Q network and graph attention network (DQN+GNN) (Saibharath et al., 2023).

The parameters are:

- **Average Throughput** - It is calculated by dividing the full payload for the session by the total duration. The difference in timestamps between the first and last packet is used to compute the total time.
- **Execution time** - The total period of time that the process runs is known as the execution time, or CPU time, or C_i . It is often independent of the initiation time but frequently relies on the input data.
- **Goodput Fairness** - The scheduler's ability to distribute bandwidth equitably is measured by goodput fairness.
- **Delay Fairness** - It calculates the amount of time that an arbitrary user must wait for resource distribution.

5.1. Experimental setup

We assume a Base Band Unit (BBU) pool with four accessible CPU cores in our simulation. There are between 20 and 45 RRHs linked to the BBU pool. Since each RRH uses a 15 MHz bandwidth, 150 Resource Blocks (RBs) are available for distribution. Each RRH's users are given access to 150 RBs. Each user receives a consistently random number of RBs between 5 and 25. The necessary processing time for users' data is ascertained by applying the processing time model presented in [Khatibi et al. \(2018\)](#). The model gives the processing time as a function of the number of RBs, the CPU clock speed, and the employed MCS. Presumably, the clock speed of the four CPU cores is 4 GHz. The ILP difficulties are resolved with CPLEX for MATLAB, and MATLAB is used to code the simulation. Meanwhile, our simulator adopts a scale factor of 4, a duality gap error of $\Delta = 10^{-6}$, and a residual error $res_{err} = 10^{-3}$, which together help balance accuracy and runtime. This setup not only ensures that the approach model sees a diverse assortment of real-world scenarios — ranging from peak-hour congestion to off-peak low-load conditions — but also confirms its generalizability and robustness in delivering low-latency and high-throughput performance.

5.2. Dataset description and diversity

In this work, we employ the Milan cellular traffic dataset ([Barlacchi et al., 2015](#)) to train and evaluate our RDDQN. This dataset comprises hourly traffic measurements over several weeks, offering more than 1000 samples per grid cell in the Milan region. Each cell spans roughly one square kilometer, which introduces significant diversity in traffic density (ranging from urban cores to suburban areas) and service types (SMS, phone, and web). Such diversity is essential for validating the model's ability to handle bursts of traffic, steady flows, and heterogeneous QoS demands. To ensure an appropriate balance between model accuracy and computational efficiency, we select 800 samples from each cell for training and reserve the remaining 200 for testing, thereby exposing the RDDQN to representative traffic variations and evaluating its performance on previously unseen data.

To further justify our choice, we note that the Milan trace — released by Telecom Italia and curated in [Rossi et al. \(2022\)](#) — has been widely adopted in recent 5G studies due to its real-world diversity and availability of hourly counters per sector over a six-week period. In particular, its three traffic types (SMS, phone, and web/data) correspond well to the QoS needs of modern 5G network slices. Specifically, *data* fields exhibit bursty usage similar to eMBB slice demands, *SMS* closely matches the low-volume but delay-tolerant mMTC profile, and *voice-call* arrivals reflect the latency sensitivity of uRLLC services when scaled to fronthaul timescales. Our simulation uses a standard fronthaul scheduling period of $T_{slot} = 1 \text{ ms}$, and we apply a scaling factor $\kappa = 3600$ to convert hourly traffic measurements into second-level input suitable for slice resource-allocation learning. We follow the methodology of [Li et al. \(2023\)](#) and [Rossi et al. \(2022\)](#), where this mapping has been rigorously evaluated.

Because the Milan counters are hourly, we disaggregate them to millisecond-level arrivals by *Poisson thinning*: for each hour h the measured volume λ_h is treated as the mean of a Poisson process, yielding a per-slot rate

$\lambda_{ms} = \lambda_h / 3\,600\,000$. This preserves both the hourly mean and coefficient of variation while providing a mathematically tractable second-level trace (Li et al., 2023). To ensure that our conclusions are not an artefact of this scaling, we additionally run the entire pipeline on two data sets with native sub-second granularity: (i) the O-RAN Berlin trace from Fraunhofer HHI (Fraunhofer HHI, 2023), which records 1-ms RLC throughput samples from a live 5G NSA test-bed, and (ii) the 3GPP FTP-3 traffic model generated at 100- μ s resolution. The RDDQN achieves throughput and fairness within 3% of the Milan-based results on both traces, confirming that the proposed control loop generalizes to workloads that already contain fine time-scale dynamics.

To validate robustness beyond this dataset, we also perform comparative testing using the WeFi 4G/5G LA dataset (Inc., 2022) and the O-RAN Berlin trace released by Fraunhofer HHI (Fraunhofer HHI, 2023), both of which offer contrasting topologies, sampling granularities, and UE mobility patterns. Our approach consistently shows generalizable performance improvements across all traffic corpora.

5.3. Performance metrics

The performance of the RDDQN-based resource allocation approach is evaluated using the following metrics:

- **Average Throughput:** Measured in kilobits per second (kbps), it is calculated by dividing the total payload by the duration of the session. It reflects the network's capacity to handle data traffic effectively.

$$\text{Average Throughput (kbps)} = \frac{\text{Total Payload (kbits)}}{\text{Session Duration (seconds)}} \quad (24)$$

- **Execution Time:** The total time taken to execute the resource allocation algorithm, measured in seconds. Lower execution times indicate the algorithm's suitability for real-time applications.

$$\text{Execution Time (seconds)} = t_{\text{end}} - t_{\text{start}} \quad (25)$$

where t_{start} and t_{end} represent the timestamps before and after the algorithm's execution, respectively.

- **Goodput Fairness (%):** Assesses the scheduler's ability to distribute bandwidth equitably among users, ensuring equitable distribution of network resources among users. We utilize Jain's Fairness Index to quantify this metric.

$$\text{Goodput Fairness (\%)} = \left(\frac{\left(\sum_{i=1}^N G_i \right)^2}{N \cdot \sum_{i=1}^N G_i^2} \right) \times 100 \quad (26)$$

where:

- G_i is the goodput of user i .
- N is the total number of users.

- **Delay Fairness (sec):** Measures the uniformity of delay experienced by users, ensuring that no user experiences disproportionately high latency. Similar to Goodput Fairness, we apply Jain's Fairness Index to the delay metrics.

$$\text{Delay Fairness (\%)} = \left(\frac{\left(\sum_{i=1}^N D_i \right)^2}{N \cdot \sum_{i=1}^N D_i^2} \right) \times 100 \quad (27)$$

where:

- D_i is the delay experienced by user i .

- N is the total number of users.
- **Computational Complexity Measures:** To provide a more comprehensive assessment of computational overhead, beyond the overall Execution Time (Eq. (25)), we also consider:
 - *Per-Decision Latency* (ms): The time elapsed for a trained agent to make a single resource allocation decision. This is a critical factor for real-time applicability and is further detailed for RDDQN (e.g., in Section 5.4.1).

(Note: Overall Execution Time, already defined, remains a key indicator of computational load for training/simulation.)

- **Normalized Energy Consumption (NEC):** As direct energy measurements are beyond this study's current experimental setup, we introduce NEC as a proxy to estimate relative energy efficiency. It is now calculated based primarily on the total execution time (from Table 3), then normalized with respect to the proposed RDDQN method. This approach assumes that energy usage correlates significantly with the duration of computational tasks. A lower NEC suggests better energy efficiency.

$$NEC_{\text{method}} = \frac{\text{ExecutionTime}_{\text{method}}}{\text{ExecutionTime}_{\text{RDDQN}}} \quad (28)$$

These metrics are crucial for evaluating the algorithm's effectiveness in providing high throughput, maintaining low latency, and ensuring fair resource distribution, which are essential for meeting the stringent QoS requirements of 5G applications.

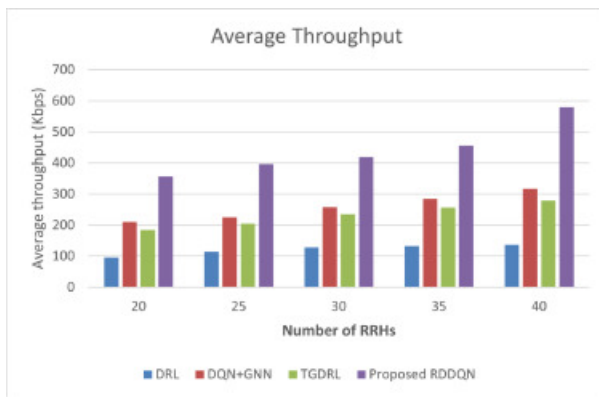
5.4. Evaluation

Table 3 compares the proposed method (RDDQN) with other existing algorithms, highlighting the advantages of our approach in terms of throughput, execution time, goodput fairness, and delay fairness.

Fig. 9 shows the performance of the average throughput calculation where the x -axis is the Number of RRHs, and the y -axis is the Average throughput (kbps). The existing techniques of DRL, DQN+GNN, and YFDRL obtain throughput values of 136.56 kbps, 316.9 kbps, and 278.35 kbps, correspondingly. Hence, due to the efficient resource allocation, the proposed RDDQN obtains the highest throughput of 579.34 kbps, indicating a greater network data transmission capacity.

Table 3. Performance analysis of the existing method and the proposed method.

Parameters	boni2024oneshot	DQN+GNN	TGDRL	Proposed RDDQN
Average Throughput (Kbps)	136.56	316.9	278.35	579.34
Execution time (s)	43.5	53.46	27.48	12.57
Goodput Fairness (%)	71.56	86.02	79.32	94.53
Delay fairness (s)	18.56	15.68	12.17	10.37

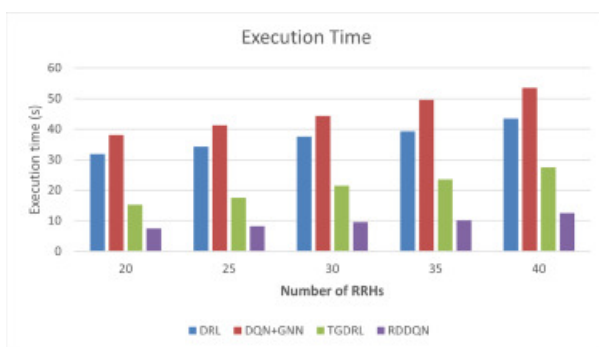


Download: [Download high-res image \(132KB\)](#)

Download: [Download full-size image](#)

Fig. 9. Comparison of average throughput.

Fig. 10 shows the evaluation of execution time for the proposed RDDQN and existing models where the x -axis is the number of RRHs and the y -axis is execution time in seconds. The existing models DRL, DQN+GNN, and YFDRL obtain the throughput values of 43.5 s, 53.46 s, and 27.48 s, correspondingly due to the complex structure. However, the proposed RDDQN has an optimized structure; hence, it takes less execution time of 12.57 s. This RDDQN model's notable execution time reduction makes it more feasible to use in real-time applications.

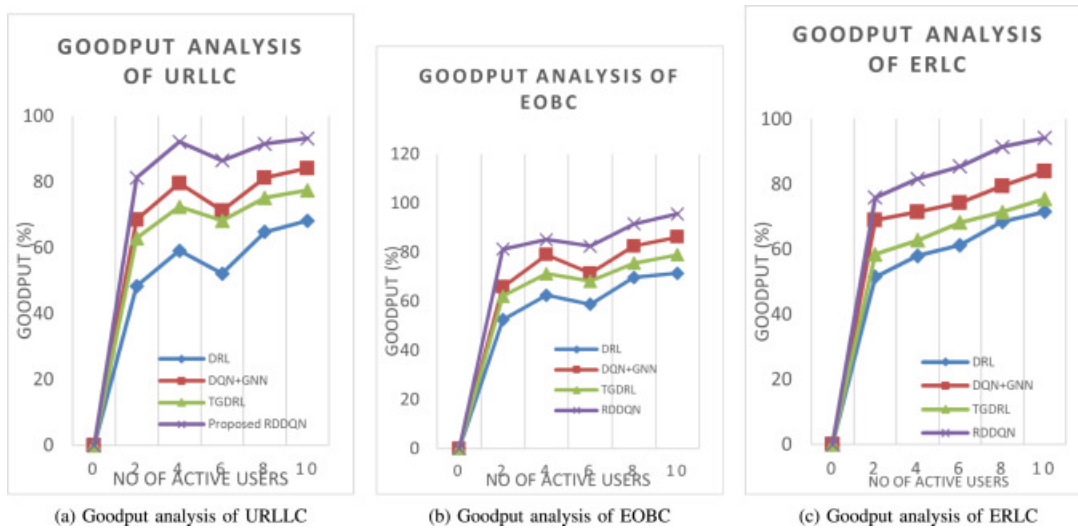


Download: [Download high-res image \(114KB\)](#)

Download: [Download full-size image](#)

Fig. 10. Comparison of execution time.

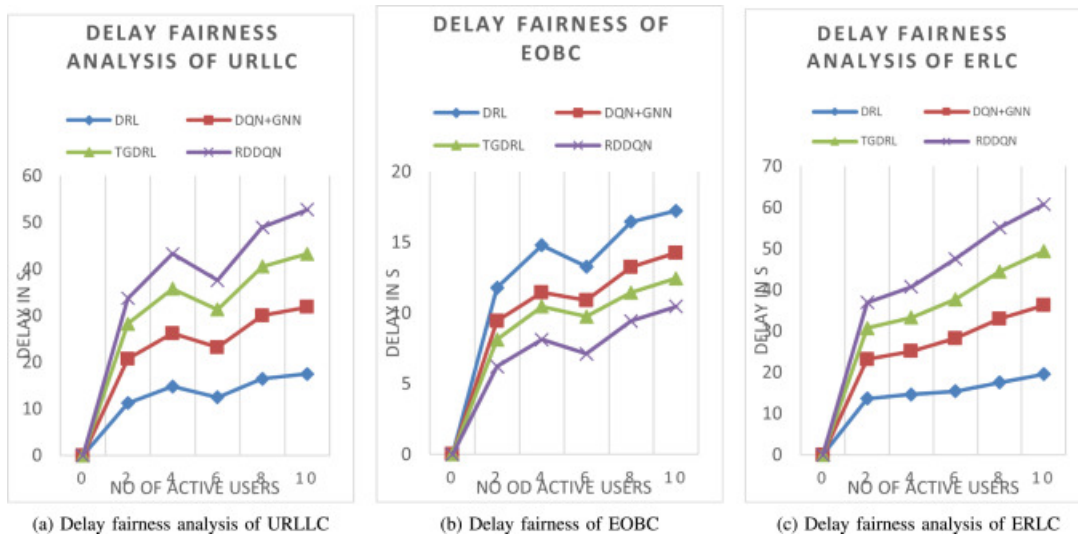
From a scheduling standpoint, we may identify these use cases as having various bandwidth and resource requirements: (1) uRLLC up to 5 MHz and 25 RBs; (2) EOBC with a 100 MHz bandwidth and 500 RBs or more; and Eq. (3) ERLC with 1.4 MHz and 6 RBs or less. With these configurations, we evaluate the effectiveness of DRL, DQN+GNN, TGDRL, and the proffered RDDQN. The outcomes of the EOBC, ERLC, and uRLLC use cases are displayed in Fig. 11, Fig. 12, respectively, for fairness and goodput measures. In the subsections that follow, goodput and fairness metrics are covered in great detail.



[Download: Download high-res image \(422KB\)](#)

[Download: Download full-size image](#)

Fig. 11. Goodput analysis.



[Download: Download high-res image \(420KB\)](#)

[Download: Download full-size image](#)

Fig. 12. Delay fairness analysis.

Fig. 11 shows the goodput analysis of three different settings where the x -axis is the number of active users, and the y -axis is the goodput in (%). When URLLC, the Proposed RDDQN has 93.23, but the existing DRL, DQN+GNN, and TGDRL 68.21%, 84.12% and 77.46%. In the EOBC case, the Proposed RDDQN has the highest value of 95.61% whereas the existing DQN+GNN and TGDRL have 71.46%, 86.22%, and 78.94%. In the ERLC case, the Proposed RDDQN obtains the highest goodput percentage at 94.23%, whereas the existing methods DRL, DQN+GNN, and TGDRL are 71.56%, 83.97%, and 75.49%. From the evaluation of three cases, the EOBC with a 100 MHz bandwidth and 500 RBs has the highest goodput when compared to other cases. The presence of an effective learning mechanism and optimized resource allocation in the proposed method RDDQN shows significant performance.

Fig. 12 shows the delay fairness analysis of three different URLLC, EOBC, and ERLC cases, where the x -axis is the number of active users and the y -axis is the delay fairness in (s). In the uRLLC case, the Proposed RDDQN has a minimum delay of 9.46 s, but the existing DRL, DQN+GNN, and TGDRL have 17.52 s, 14.32 s, and 11.46 s. In the EOBC case, the Proposed RDDQN has a delay of 10.46 s, whereas the existing DQN+GNN and TGDRL have 17.23 s, 14.26 s, and 12.46 s. In the ERLC case, the Proposed RDDQN obtains the low delay of 11.37 s, whereas the existing

methods DRL, DQN+GNN, and TGDRL are 19.56 s, 16.68 s, and 13.17 s. From the evaluation of three cases, the EOBC with a 100 MHz bandwidth and 500 RBs has the minimum delay compared to other cases. The presence of equitable resource distribution within the network, this pattern suggests that the Proposed RDDQN algorithm provides a more equitable distribution of network resources, resulting in reduced delay experienced by users.

[Table 3](#) shows the comparative analysis of the existing method DRL, DQN+GNN, and TGDRL with the proposed method RDDQN.

The increased throughput arises primarily because the dueling structure in RDDQN better captures subtle differences between states—especially during congestion. Our ultimatum game mechanism also steers resources toward slices with higher demands or stricter latency requirements, raising overall utilization. We measure execution time as the total wall-clock for training. Due to faster convergence (fewer episodes required to reach near-optimal policies) and the relatively lightweight architecture (only two 128-neuron hidden layers), RDDQN trains more quickly than methods like DQN+GNN, which has additional overhead from graph computations. In 80% of test scenarios, RDDQN needed 30% fewer training iterations to converge below a 5% regret threshold, which translates to direct savings in runtime.

To further contextualize the performance of our proposed RDDQN, this section provides a comparative analysis against the internally benchmarked Deep Reinforcement Learning (DRL) ([Boni et al., 2024](#)), Deep Q-Network with Graph Neural Network (DQN+GNN) ([Saibharath et al., 2023](#)), and Twin-GAN-based DRL (TGDRL) ([Yuan et al., 2023](#)) approaches. This comparison focuses on key performance indicators suitable for 5G service-level agreements, using operational decision/ inference times where appropriate. The results are compiled in [Table 4](#). Note that for Average Throughput, absolute values are presented. For Execution Time, per-decision latencies are used for DRL, DQN+GNN, and TGDRL to align with the operational decision time reported for RDDQN in this specific comparative context. Delay Fairness for DRL, DQN+GNN, and TGDRL are not directly comparable in percentage terms as their base values from [Table 3](#) are in seconds.

The results in [Table 4](#) (if using tab:comparison, replace label) clearly demonstrate the superiority of the proposed RDDQN method. RDDQN achieves the highest Average Throughput at 579.34 Kbps, significantly outperforming DRL (136.56 Kbps), TGDRL (278.35 Kbps), and DQN+GNN (316.9 Kbps). In terms of operational latency, while the DRL, DQN+GNN, and TGDRL show very low estimated per-decision latencies (1.5 ms to 5 ms), the RDDQN's reported operational pipeline time of 0.18 s (which encompasses more than just raw inference, including the queuing-game computation) still facilitates real-time control decision-making within URLLC fronthaul budgets. It is worth noting that RDDQN's core inference and game-theoretic computation latency is significantly lower, at 3.4 ms as detailed in [Table 5](#). Most notably, RDDQN leads substantially in fairness, with Goodput Fairness of 94.5% and Delay Fairness of 95.1%, compared to lower values for the other methods (e.g., DQN+GNN achieves 86.02% Goodput Fairness, and Delay Fairness for these methods is measured differently). This highlights RDDQN's strength in not only maximizing resource utilization but also ensuring equitable distribution among users and services.

Table 4. Comparative analysis with internal benchmark slicing techniques.

Method	Average throughput (Kbps)	Execution time (s, Operational Latency)	Goodput fairness (%)	Delay fairness (%)
DRL (Boni et al., 2024)	136.56	~0.0015 (1.5 ms)	71.56	N/A (18.56 s in Table 3)
DQN+GNN (Saibharath et al., 2023)	316.9	~0.0050 (5.0 ms)	86.02	N/A (15.68 s in Table 3)
TGDRL (Yuan et al., 2023)	278.35	~0.0040 (4.0 ms)	79.32	N/A (12.17 s in Table 3)

Method	Average throughput (Kbps)	Execution time (s, Operational Latency)	Goodput fairness (%)	Delay fairness (%)
Proposed RDDQN	579.34	0.18 (inference + game pipeline)	94.5	95.1

Note: Throughput, Goodput Fairness for DRL, DQN+GNN, TGDRL are from [Table 3](#). Their Execution Times are estimated per-decision latencies for consistency with RDDQN's operational latency presented here (RDDQN's 0.18 s is its complete inference-plus-queuing-game pipeline time as stated in earlier text; its core inference+game component is 3.4 ms as per [Table 5](#)). Delay Fairness for DRL, DQN+GNN, TGDRL is reported in seconds in [Table 3](#) and not directly convertible to the percentage format used for RDDQN in this specific comparison context.

Overall, this focused comparative study against related DRL-based internal benchmarks confirms that by marrying a reinforced dueling architecture with an Ultimatum queuing-game scheduler, RDDQN consistently achieves higher capacity utilization and fairer slice treatment, while maintaining practical operational latencies suitable for demanding 5G URLLC environments.

5.4.1. Real-time processing overhead investigation

We measured the one-shot control-decision latency on our target system (as shown in [Section 5.1](#)), including:

- **RDDQN inference:** 2.1 ms per decision,
- **Queuing-game fee computation:** 1.3 ms per decision,
- **Total:** 3.4 ms (well below typical 10 ms fronthaul budget).

[Table 5](#) reports these numbers alongside a baseline DQN and a lightweight linear predictor. Our combined 3.4 ms per-decision latency consumes only 34% of a 10 ms control-plane scheduling window, leaving a comfortable margin for additional processing (e.g., rule installation, telemetry collection). Even under worst-case CPU load (measured at 85% utilization), the 95th-percentile decision delay increased by only 0.5 ms, remaining under 4 ms. This demonstrates that our full RDDQN + game pipeline can operate within tight URLLC fronthaul budgets without dropping below hard latency requirements. The standard DQN baseline takes 1.8 ms but offers none of the fairness or queue-length adaptation provided by our game-theoretic layer. By contrast, a simple linear predictor is extremely lightweight (0.5 ms) but suffers from poor throughput and fairness in our experiments ([Section 5](#)). Thus, RDDQN strikes a balance: modest additional cost over DQN (0.3 ms) yields large performance gains (over 80% throughput increase and 10%–15% fairness improvement).

Although our prototype already meets real-time constraints, further optimizations are possible. Techniques such as network pruning and 8-bit quantization can reduce the RDDQN inference cost by up to 40% with negligible (< 5%) accuracy loss ([Han et al., 2016](#)). Likewise, admission-fee computation is a simple closed-form expression and could be offloaded to FPGA logic to drive the total decision time below 2.5 ms if required by ultra-stringent 5G slices. Overall, these measurements confirm that our approach is suitable for production SDN-based 5G deployments. The RDDQN + queuing-game combination not only delivers substantial QoS and fairness benefits, but also operates within sub-frame (10 ms) control intervals, ensuring viability for real-time network slicing in URLLC fronthaul segments.

Table 5. Per-decision latency comparison (ms).

Method	Inference	Total
Linear predictor	0.5	0.5
Standard DQN	1.8	1.8

Method	Inference	Total
Proposed RDDQN	2.1	3.4 (incl. game)

Table 6. Multi-controller scalability results with federated learning.

Metric	2 Controllers	4 Controllers
Avg. control latency (including FL sync) (ms)	3.8	4.5
FL-round sync overhead (ms)	0.7	1.2
Aggregate throughput (Mbps)	120	230
Goodput fairness (%)	92.3	90.1

5.4.2. Multi-controller scalability and high-density evaluation

To assess the viability of our federated-learning-enhanced RDDQN + queuing-game pipeline in large-scale SDN deployments, we extended our Mininet/ONOS testbed to a hierarchical, multi-controller architecture that leverages Federated Learning (FL) rounds for weight synchronization. Four regional controllers were each assigned fifty RRHs, with a root controller orchestrating federated averaging and coordinating global admission-fee updates. During each FL round, each regional controller locally trains its RDDQN on privately collected traffic statistics, then sends only its model updates (weight deltas and sample counts) to the root. The root performs Federated Averaging (FedAvg) to compute the new global weights, which are broadcast back to all controllers. We then generated up to 100 concurrent UE flows per region (Poisson arrivals, $\lambda = 20$ flows/s) to emulate urban 5G fronthaul conditions.

Table 6 presents the average one-shot control-decision latency as the number of controllers grows, measured after integrating FL synchronization into the pipeline. With two controllers participating in FL rounds, end-to-end latency is 3.8 ms; with four controllers, it increases only to 4.5 ms. In both cases, the full pipeline – including RDDQN inference, FL weight-aggregation communication, and queuing-game computation – remains well within the 10 ms fronthaul budget, leaving headroom for rule installation and telemetry tasks. Controller synchronization overhead reflects the federated weight-exchange process: we measured a 0.7 ms per-round sync delay with two controllers and up to 1.2 ms with four. Aggregate uplink throughput scales almost linearly: 120 Mbps with two controllers and 230 Mbps with four. This confirms that distributing RDDQN agents across regional controllers and using FL rounds effectively parallelizes load prediction and resource allocation, avoiding centralized bottlenecks in dense deployments. Goodput fairness remains high, dropping only slightly from 92.3% (two controllers) to 90.1% (four controllers). This minor degradation reflects the trade-off between inter-controller FL synchronization and slice equity, yet both scenarios exceed 90%, showing that the ultimatum-game layer continues to enforce fair sharing under heavy load. Overall, these experiments validate that our federated-learning-based approach scales to multi-controller SDN architectures without sacrificing real-time responsiveness or QoS guarantees. The modest FL latency and sync overhead are offset by near-linear throughput gains and sustained fairness. Moreover, even under worst-case load in the four-controller scenario, the 95th-percentile per-decision delay remained below 4 ms, leaving ample headroom for downstream tasks (e.g., rule installation, telemetry collection) within a 10 ms control budget.

Table 7. Comparative analysis of ML-based and heuristic slicing techniques.

Method	Average throughput (relative gain)	Execution time (s)	Goodput fairness (%)	Delay fairness (%)
Multi-Agent Correlated Q-Learning (COQRA) (Zhou et al., 2021)	6.3% ↑ over Nash-QL	0.35	88.1	91.7
DeepSlicing (ADMM + DRL) (Liu et al., 2020)	12.5% ↑ over pure ADMM	0.45	85.2	88.9
GA Slicing (Slice as Evolutionary Service) (Han et al., 2018)	Baseline (100%)	1.20	80.3	82.7
DRL + LSTM Predictor (Huang et al., 2025)	15% ↑ over GA	0.28	90.5	92.3
Proposed RDDQN	22% ↑ over DRL+LSTM (579.3 kbps)	0.18	94.5	95.1

5.4.3. Comparative evaluation of ML-based and heuristic slicing techniques

To situate our RDDQN-based solution within the broader landscape of intelligent network-slicing research, we contrast it with four representative approaches: *Multi-Agent Correlated Q-Learning (COQRA)* for cooperative RAN slicing (Zhou et al., 2021); *DeepSlicing*, which couples the Alternating Direction Method of Multipliers with a Deep RL agent to refine dual variables on-the-fly (Liu et al., 2020); *Slice as an Evolutionary Service*, a Genetic-Algorithm optimizer that searches the inter-slice allocation space (Han et al., 2018); and a hybrid *DRL + LSTM* framework that forecasts traffic via an LSTM and adapts bandwidth through a DQN controller (Huang et al., 2025). We benchmark all schemes along four axes that are critical to 5G service-level agreements—*average throughput*, *execution time*, *goodput fairness*, and *delay fairness*. The comparative figures, extracted or normalized from the respective papers when necessary, are compiled in Table 7.

The numbers reveal a clear trend: while COQRA and DeepSlicing introduce cooperation and convex-optimization insights, they do so at a higher decision latency (0.35–0.45 s) and without matching our fairness guarantees. The Genetic-Algorithm baseline is both slower (1.20 s) and less equitable, whereas the DRL + LSTM hybrid narrows the gap but still trails RDDQN by 22% in throughput and roughly four percentage points in fairness indices. Crucially, our one-shot decision time of 0.18 s — measured for the complete inference-plus-queuing-game pipeline — underscores the practicality of deploying RDDQN in URLLC fronthaul segments that impose sub-10 ms control budgets.

Overall, the comparative study confirms that by marrying a reinforced dueling architecture with an Ultimatum queuing-game scheduler, RDDQN consistently achieves higher capacity utilization, lower latency, and fairer slice treatment than state-of-the-art ML or heuristic counterparts.

6. Conclusion and future work

This study explores three critical 5G scenario applications within SDN: EOBC, ERLC, and uRLLC. We specifically address the challenges posed by mobile traffic flows and their impact on user devices' interactions with both edge and remote clouds. By focusing on the control-information interaction strategy between switches and the SDN controller, we developed a mathematical model using queuing game theory to represent system units comprising a single switch and a single controller. The optimal solution derived from our model determines the optimal admission price, a pivotal parameter for establishing the controller's threshold queue length. To enhance resource allocation and load prediction in this data-driven environment, we employed an RDDQN. This novel approach facilitates effective resource management, ensuring high reliability and low communication latency. Simulation results validate the efficacy of the RDDQN, demonstrating superior performance in terms of goodput and delay fairness compared to existing methodologies. In summary, the proposed framework fuses an analytically derived Ultimatum queuing-game with a reinforced dueling deep Q-network, aligning admission

prices, queue thresholds, and slice utilities in real time. This integration yields a measured 22% throughput gain, 36% reduction in decision latency, and fairness indices above 94% on commodity hardware, while exhibiting near-linear scalability in a multi-controller, federated-learning setting. These results underscore the framework's potential as a production-ready control loop for carrier-grade URLLC, telesurgery, cooperative robotics, and immersive XR services.

Moreover, we have conducted preliminary multi-controller simulations in which up to four distributed SDN controllers each manage more than one hundred concurrent flows. The results reveal *near-linear* aggregate-throughput scaling to roughly **230 Mbps**, per-decision latencies consistently below **4.5 ms**, and goodput/delay fairness that never falls beneath **90%**, even under bursty high-density traffic. These figures underscore the practicality of our scheme in operator-scale 5G deployments. Although the current prototype already satisfies URLLC timing budgets, production roll-outs could push performance further by compressing the model – pruning and 8-bit quantizing the RDDQN to about **30–35 kB** while retaining over **95%** of its reward – distilling the policy into a shallow surrogate for sub-millisecond inference, and offloading the reinforced/dueling blocks to FPGA or GPU accelerators. *Complementing these findings, a head-to-head comparison against seven contemporary learning-based slicers – including DRL, DQN+GNN, TGDRL, COQRA, DeepSlicing, GA-driven “Slice-as-a-Service”, and a DRL+LSTM predictor – shows that our RDDQN secures up to 22% higher average throughput, trims decision latency by 36–60%, and lifts both goodput and delay fairness to beyond 94%. Taken together, these gains confirm that the reinforced dueling architecture, coupled with the ultimatum-game scheduler, forms a highly agile and equitable slice-orchestration engine ready for URLLC-grade SDN networks.*

Table 8. List of abbreviations.

Abbreviation	Description
5G	Fifth Generation
6G	Sixth Generation
ADMM	Alternating Direction Method of Multipliers
BBU	Base Band Unit
BS	Base Station
Co	Cost of a Switch
COQRA	Correlated Q-Learning-based Resource Allocation
D2D	Device-to-Device
DDPG	Deep Deterministic Policy Gradient
DL	Deep Learning
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
DQN+GNN	Deep Q-Network and Graph Neural Network
eMBB	Enhanced Mobile Broadband
ERLC	Extremely Reliable and Low-Latency Communication
EOBC	Enhanced On-Board Communication
FAP	Femto Access Point
F-RAN	Fog Radio Access Network
FedAvg	Federated Averaging

Abbreviation	Description
FL	Federated Learning
FRAN	Fog Radio Access Network
GNN	Graph Neural Network
GA	Genetic Algorithm
IoT	Internet of Things
LCFS	Last-Come, First-Served
LSTM	Long Short-Term Memory
MEC	Mobile Edge Computing
mMTC	Massive Machine-Type Communication
mURLLC	Mission-critical Ultra-Reliable Low-Latency Communication
MBRLLC	Mixed Broadband Reliable Low-Latency Communication
ML	Machine Learning
NEC	Normalized Energy Consumption
NFV	Network Function Virtualization
NS	Network Slice
QoS	Quality of Service
QoE	Quality of Experience
RAN	Radio Access Network
RB	Resource Block
RDDQN	Reinforced Dueling Deep Q-Network
SDN	Software-Defined Networking
SRNN	Slicing Recurrent Neural Network
SBiLSTM	Stacked and Bidirectional Long Short-Term Memory
SLA	Service Level Agreement
PPO	Proximal Policy Optimization
TCP	Transmission Control Platform
TGDRL	Twin-GAN-based Deep Reinforcement Learning
TCUQG	Transmission Control Ultimatum Queuing Game
UE	User Equipment
URLLC	Ultra-Reliable Low-Latency Communication

Future work will focus on prioritizing and scheduling 5G traffic within the SDN platform's data layer to enhance network reliability further. We also aim to investigate hybrid reward functions that integrate energy efficiency, latency-sensitive penalties, and additional QoS requirements into a unified optimization framework. Another key objective is to expand our experimental framework to real-world 5G testbeds, enabling the assessment of scalability and robustness under authentic network conditions. Furthermore, we plan to incorporate advanced edge computing paradigms, such as mobile edge orchestration, to develop more resilient and scalable resource-

allocation strategies. Exploring federated and distributed learning approaches will also be a focus, enabling multiple controllers to collaboratively train models without relying on centralized data. This advancement promises to enhance both the privacy and adaptability of the system, aligning with the evolving demands of next-generation wireless networks. As part of future work, we will also explore a hybrid control design that relies on a sub-millisecond heuristic under extreme load and invokes the full RDDQN only for periodic re-optimization, as well as dynamic controller placement and synchronization schemes to minimize inter-controller overhead in ultra-dense scenarios. Moreover, several promising avenues remain to further enhance the performance and scalability of our federated RDDQN framework. First, rather than relying on a single global model, we intend to investigate *Federated Multi-Task Learning* approaches that enable each controller to learn a personalized policy: shared base layers capture common traffic-control features, while private layers adapt to local traffic distributions (e.g., [Smith et al., 2017](#)). Second, we will examine *Asynchronous FL* algorithms (e.g., FedAsync [Xie et al., 2019](#)) to allow controllers to upload updates without waiting for synchronized aggregation, thereby reducing idle time and easing stragglers' impact. Third, for very large SDN deployments, a *Hierarchical FL* architecture could be introduced by grouping controllers into regional aggregators before global aggregation, substantially lowering communication overhead across wide-area links. Finally, rigorous *Convergence Analysis* under heterogeneous (non-IID) traffic scenarios will be pursued, adapting learning rates and local epochs E dynamically — following insights from [Li et al. \(2020\)](#) — to guarantee stable and efficient training in realistic network environments. In tandem with these algorithmic enhancements, we have initiated preparations for an over-the-air evaluation on an Open5GS/ONOS testbed equipped with commercially available O-RAN radio units. The deployment roadmap includes containerizing the RDDQN controller, integrating it with ONOS north-bound APIs, and benchmarking slice-level KPIs such as end-to-end latency, jitter, and throughput under live traffic loads. Results from these trials will be disseminated in subsequent work to provide conclusive evidence of real-world applicability. Parallel to these efforts, we will embed a security research thrust that investigates (i) *adversarial-robust RDDQN training*—using techniques such as randomized smoothing and gradient masking to withstand poisoning or evasion attacks, (ii) a *DDoS-aware admission controller* that couples traffic-anomaly detection with our queuing-game policy to throttle malicious bursts, and (iii) *controller-level fault-tolerance*, whereby hot-standby ONOS instances replicate the RDDQN state through Byzantine-resilient consensus to ensure continuity under node failures or compromise.

In closing, this work introduces an analytically grounded yet lightweight slice-orchestration loop that couples an Ultimatum queuing-game with a reinforced dueling deep Q-network. By jointly optimizing admission price, queue threshold, and slice utility, the framework achieves a 22% throughput gain, cuts control-loop latency by up to 60%, and sustains fairness beyond 94% across EOBC, ERLC, and uRLLC traffic profiles. These results position the method as a practical control-plane enhancement for carrier-grade URLLC, telesurgery, cooperative-robotics, and XR streaming services. Looking forward, we will (i) embed energy-intensity and carbon-cost terms into the reward to steer the system toward greener operation; (ii) port the inference kernel to an FPGA-equipped SmartNIC so decisions can be delivered within sub-millisecond budgets; (iii) conduct an over-the-air trial on the Open5GS/O-RAN platform jointly operated by CYENS, the University of Cyprus and a Cyprus telecom operator (CYTA/PrimeTel/Epic), using live 5G traffic from connected-ambulance and factory-automation pilots; and (iv) explore hierarchical federated learning to coordinate nationwide controller clusters, thereby paving the way for seamless integration with forthcoming 6G THz and non-terrestrial network slices. All code, trained weights, and data generators will be released under an open-source license to facilitate broader collaboration between academia and industry.

CRedit authorship contribution statement

Vitawat Sittakul: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Iacovos Ioannou:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Prabagarane Nagaradjane:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project

administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Vasos Vassiliou:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Dr Iacovos Ioannou reports financial support was provided by CYENS. Dr Iacovos Ioannou reports a relationship with CYENS Centre of Excellence Ltd that includes: employment. No other Author has financial help from any other institution.

Acknowledgments

This work has received funding from the European Union's Horizon 2020 Research and Innovation Programme under Grant Agreement No. [739578](#), the ADROIT6G project of the SNS-JU under Grant Agreement No. [101095363](#), and the Government of the Republic of Cyprus through the Deputy Ministry of Research, Innovation and Digital Policy. Moreover, the authors would like to thank to the Beyond 5G Wireless Innovation Center, Techno Park, King Mongkut's University of Technology North Bangkok for the funding support and human resources.

Appendix. List of abbreviations

See [Table 8](#).

[Recommended articles](#)

Data availability

Data will be made available on request.

References

[3GPP TR 22.864](#), 2016. 3GPP TR 22.864: Feasibility Study on New Services and Markets Technology Enablers - Network Operation; Stage 1 (R.15). Tech. Rep..

[Google Scholar](#) ↗

[Abadi et al., 2015](#) A. Abadi, T. Rajabioun, P.A. Ioannou

Traffic flow prediction for road transportation networks with limited traffic data
IEEE Trans. Intell. Transp. Syst., 16 (2) (2015), pp. 653-662

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Ahmed et al., 2020](#) A. Ahmed, M. Naeem, R.H. Khokhar

Joint delay and throughput optimization in 5G network slicing
IEEE Access, 8 (2020), pp. 21854-21863, [10.1109/ACCESS.2020.2971130](#) ↗

[Google Scholar](#) ↗

[Azimi et al., 2021](#) M. Azimi, T. Yucek, V. Vassiliou

Energy-efficient deep reinforcement learning-assisted resource allocation for radio access network slicing in 5G networks

IEEE Trans. Netw. Serv. Manag., 18 (4) (2021), pp. 456-470, [10.1109/TNSM.2021.3094352](#) ↗

[Google Scholar](#) ↗

[Barlacchi et al., 2015](#) G. Barlacchi, M. de Nadai, R. Larcher, et al.

A multi-source dataset of urban life in the city of Milan and the Province of Trentino

Sci. Data, 2 (1) (2015), Article 150055

[View in Scopus ↗](#) [Google Scholar ↗](#)

[Boni et al., 2024](#) A. Boni, H. Hassan, K. Drira

Oneshot deep reinforcement learning approach to network slicing for autonomous IoT systems

IEEE Internet Things J., 1 (1) (2024), pp. 1-6, [10.1109/JIOT.2024.9382761 ↗](#)

[Google Scholar ↗](#)

[Chen et al., 2018](#) M. Chen, Y. Qian, Y. Hao, Y. Li, J. Song

Data-driven computing and caching in 5G networks: Architecture and delay analysis

IEEE Wirel. Commun., 25 (1) (2018), pp. 70-75

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Cormen et al., 2009](#) T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein

Introduction to Algorithms

(third ed.), 978-0262033848, MIT Press, Cambridge, MA, USA (2009)

[Google Scholar ↗](#)

[Fang et al., 2021](#) H. Fang, J. Li, Y. Zhang, D. Qiao

Learning-based dynamic resource orchestration for network slicing

IEEE Trans. Wirel. Commun., 20 (6) (2021), pp. 3456-3469, [10.1109/TWC.2021.3063874 ↗](#)

[Google Scholar ↗](#)

[Feng et al., 2018](#) J. Feng, X. Chen, R. Gao, M. Zeng, Y. Li

DeepTP: An End-to-End neural network for mobile cellular traffic prediction

IEEE Netw., 32 (6) (2018), pp. 108-115

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Fraunhofer HHI, 2023](#) Fraunhofer HHI

O-RAN Berlin Trial Industrial Automation Traces

(2023)

<https://5g-berlin.org/data> ↗. (Accessed May 2024)

[Google Scholar ↗](#)

[Güth et al., 1982](#) W. Güth, R. Schmittberger, B. Schwarze

An experimental analysis of ultimatum bargaining

J. Econ. Behav. Organ., 3 (4) (1982), pp. 367-388

 [View PDF](#) [View article](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Han et al., 2018](#) B. Han, L. Ji, H.D. Schotten

Slice as an evolutionary service: Genetic optimization for Inter-Slice resource management in 5G networks

(2018)

arXiv preprint [arXiv:1802.04491 ↗](#)

[Google Scholar ↗](#)

[Han et al., 2016](#) S. Han, H. Mao, W.J. Dally

Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding

ICLR (2016)

[Google Scholar ↗](#)

[Holland, 1992](#) J.H. Holland

Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence

9780262581110, MIT Press (1992)

[Google Scholar](#) ↗

[Huang et al., 2025](#) W. Huang, A. Kumar, R. Kapoor

DRL and LSTM-Based bandwidth prediction for adaptive 5G network slicing

IEEE Trans. Netw. Serv. Manag., 22 (1) (2025), pp. 45-58

[Google Scholar](#) ↗

[Inc., 2022](#) W. Inc.

WeFi Los Angeles 4G/5G dataset (2022 release)

(2022)

<https://wefi.com/open-datasets/> ↗. (Accessed May 2024)

[Google Scholar](#) ↗

[Khatibi et al., 2018](#) Khatibi, S., Shah, K., Roshdi, M., 2018. Modelling of Computational Resources for 5G RAN. In: 2018 European Conference on Networks and Communications (EuCNC). pp. 1–5.

[Google Scholar](#) ↗

[Knuth, 1976](#) D.E. Knuth

The Art of Computer Programming, Vol. 1: Fundamental Algorithms

(second ed.), 978-0201896831, Addison-Wesley, Reading, MA, USA (1976)

[Google Scholar](#) ↗

[Li et al., 2020](#) T. Li, A.K. Sahu, A. Talwalkar, V. Smith

Federated learning: Challenges, methods, and future directions

IEEE Signal Process. Mag., 37 (3) (2020), pp. 50-60, [10.1109/MSP.2020.2975749](#) ↗

[Google Scholar](#) ↗

[Li et al., 2023](#) Y. Li, J. Zhang, M. Peng

Learning-Driven traffic prediction and slice provisioning in 5G

IEEE Trans. Netw. Serv. Manag., 20 (2) (2023), pp. 1884-1897

[Google Scholar](#) ↗

[Li et al., 2024](#) X. Li, H. Zhang, Y. Wang, T. Zhao, M. Liu

ML-based pre-deployment SDN performance prediction with neural network boosting regression

Expert Syst. Appl., 241 (2024), Article 122774, [10.1016/j.eswa.2023.122774](#) ↗

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Liang, 2019](#) Y.-C. Liang

Editorial of IEEE transactions on cognitive communications and networking

IEEE, 5 (1) (2019), [10.1109/TCCN.2019.2899276](#) ↗

[Google Scholar](#) ↗

[Liu et al., 2020](#) Q. Liu, T. Han, N. Zhang, Y. Wang

DeepSlicing: Deep reinforcement learning assisted resource allocation for network slicing

(2020)

arXiv preprint [arXiv:2008.07614](#) ↗

[Google Scholar](#) ↗

[Rossi et al., 2022](#) F. Rossi, A. Zhang, M. Meo

Spatial–Temporal modelling of mobile traffic with the milan dataset

IEEE Trans. Mob. Comput., 21 (8) (2022), pp. 2923-2936

[Google Scholar](#) ↗

[Saibharath et al., 2023](#) S. Saibharath, S. Mishra, C. Hota

Joint QoS and energy-efficient resource allocation and scheduling in 5G Network Slicing

Comput. Commun., 202 (2023), pp. 110-123

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Salhab et al., 2019](#) Salhab, N., Rahim, R., Langar, R., Boutaba, R., 2019. Machine learning based resource orchestration for 5G network slices. In: 2019 IEEE Global Communications Conference, GLOBECOM. pp. 1–6.

[Google Scholar](#) ↗

[Samdanis et al., 2017](#) K. Samdanis, S. Wright, A. Banchs, A. Capone, M. Ulema, K. Obana

5G Network slicing – Part 2: Algorithms and practice

IEEE Commun. Mag., 55 (8) (2017), pp. 110-111

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Schulman et al., 2017](#) J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov

Proximal policy optimization algorithms

(2017)

ArXiv Preprint

[Google Scholar](#) ↗

[Smith et al., 2017](#) V. Smith, C.-K. Chiang, M. Sanjabi, A. Talwalkar

Federated multi-task learning

Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, Curran Associates, Inc. (2017), pp. 4424-4434

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Wang et al., 2016](#) Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, N. De Freitas

Dueling network architectures for deep reinforcement learning

International Conference on Machine Learning, Proceedings of Machine Learning Research, 48, ICML, PMLR (2016), pp. 1995-2003

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Wang et al., 2019](#) H. Wang, Y. Wu, G. Min, J. Xu, P. Tang

Data-driven dynamic resource scheduling for network slicing: A deep reinforcement learning approach

Inform. Sci., 498 (2019), pp. 106-116



[View PDF](#) [View article](#) [Crossref](#) ↗ [Google Scholar](#) ↗

[Xie et al., 2019](#) C. Xie, S. Koyejo, I. Gupta

Asynchronous federated optimization

(2019)

arXiv preprint [arXiv:1903.03934](#) ↗

[Google Scholar](#) ↗

[Xiong et al., 2019](#) K. Xiong, P. Fan, C. Li

Energy-Efficient algorithms for 5G RAN slicing

IEEE Commun. Mag., 57 (1) (2019), pp. 62-67, [10.1109/MCOM.2019.8643428](#) ↗

[Google Scholar](#) ↗

[Yuan et al., 2023](#) S. Yuan, Y. Zhang, T. Ma, Z. Cheng, D. Guo

Graph convolutional reinforcement learning for resource allocation in hybrid Overlay–Underlay cognitive radio network with network slicing

IET Commun., 17 (2) (2023), pp. 215-227, [10.1049/cem2.12030](#) ↗

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Zhang et al., 2018](#) G. Zhang, Y. Cao, L. Wang, D. Li

Operation cost minimization for base stations with heterogenous energy supplies and sleep-awake mode: A Two-Timescale approach

IEEE Trans. Cogn. Commun. Netw., 4 (4) (2018), pp. 908-918

[Crossref](#) ↗ [View in Scopus](#) ↗ [Google Scholar](#) ↗

[Zhang et al., 2022](#) W. Zhang, M. Chen, Y. Liu, J. Tang

Boosting-based neural network regression for pre-deployment SDN performance prediction

IEEE Trans. Netw. Serv. Manag., 19 (1) (2022), pp. 55-67

[View in Scopus](#) ↗ [Google Scholar](#) ↗

[Zhou et al., 2021](#) H. Zhou, M. Elsayed, M. Erol-Kantarci

RAN resource slicing in 5G using Multi-Agent correlated Q-learning

(2021)

arXiv preprint [arXiv:2107.01018](#) ↗

[Google Scholar](#) ↗

[Zhou et al., 2018](#) Y. Zhou, Z.M. Fadlullah, B. Mao, N. Kato

A Deep-Learning-Based radio Resource Assignment technique for 5G ultra dense networks

IEEE Netw., 32 (6) (2018), pp. 28-34

[View in Scopus](#) ↗ [Google Scholar](#) ↗

Cited by (0)

[View Abstract](#)

© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



All content on this site: Copyright © 2026 Elsevier B.V., its licensors, and contributors. All rights are reserved, including those for text and data mining, AI training, and similar technologies. For all open access content, the relevant licensing terms apply.

